

Dynamic Collision Avoidance Using Velocity Obstacle-Based Control Barrier Functions

Jihao Huang^{1b}, Jun Zeng^{2b}, Xuemin Chi, Koushil Sreenath^{3b}, Zhitao Liu^{4b}, *Member, IEEE*,
and Hongye Su^{5b}, *Senior Member, IEEE*

Abstract—Designing safety-critical controllers for acceleration-controlled unicycle robots is challenging, as control inputs may not appear in the constraints of control Lyapunov functions (CLFs) and control barrier functions (CBFs), leading to invalid controllers. Existing methods often rely on state-feedback-based CLFs and high-order CBFs (HOCBFs), which are computationally expensive to construct and fail to maintain effectiveness in dynamic environments with fast-moving, nearby obstacles. To address these challenges, we propose constructing velocity obstacle (VO)-based CBFs (VOCBFs) in the velocity space to enhance dynamic collision avoidance capabilities, instead of relying on distance-based CBFs that require the introduction of HOCBFs. Additionally, by extending VOCBFs using variants of VO, we enable reactive collision avoidance between robots. We formulate a safety-critical controller for acceleration-controlled unicycle robots as a mixed-integer quadratic programming (MIQP), integrating state-feedback-based CLFs for navigation and VOCBFs for collision avoidance. To enhance the efficiency of solving the MIQP, we split the MIQP into multiple suboptimization problems and employ a decision network to reduce computational costs. Numerical simulations demonstrate that our approach effectively guides the robot to its target while avoiding collisions. Compared to HOCBFs, VOCBFs exhibit significantly improved dynamic obstacle avoidance performance, especially when obstacles are fast moving and close to the robot. Furthermore, we extend our method to distributed multirobot systems.

Index Terms—Control barrier function (CBF), control Lyapunov function (CLF), safety-critical control, velocity obstacle (VO).

I. INTRODUCTION

A. Motivation

WITH advancements in robotics, robots are increasingly used in applications such as autonomous driving, delivery services, and industrial production [1]. Ensuring reliable

collision avoidance is essential for these applications to ensure safety and prevent potential losses. Dynamic collision avoidance, which requires considering both the positions and the velocities of obstacles, has gained significant attention. Recently, control barrier function (CBF)-based approaches are widely used in safety-critical controllers [2], [3], [4], [5], [6], [7], [8], [9], [10], which prioritize safety over other aspects, such as tracking. When designing CBFs for acceleration-controlled unicycle robots based on position and Euclidean distance, it is necessary to introduce high-order CBFs (HOCBFs) [11] to ensure that all control inputs explicitly appear in the CBF constraints. However, designing appropriate HOCBFs is computationally expensive and is less effective for dynamic obstacle avoidance. Therefore, to avoid the use of HOCBFs and achieve better dynamic obstacle avoidance performance, designing CBFs in the velocity space is an effective solution. The velocity obstacle (VO) [12]-based methods are widely used for local collision avoidance between circular robots and obstacles by selecting velocities outside the VO, with several works [13] showing promising results by formulating VO-based obstacle avoidance constraints in optimization problems. To achieve better dynamic obstacle avoidance performance while ensuring real-time capability, we propose constructing VO-based CBFs (VOCBFs). Additionally, to mitigate VO deadlocks and enable navigation for acceleration-controlled robots, we design state-feedback-based control Lyapunov functions (CLFs). By combining VOCBFs and CLFs, the robot can move toward its target while ensuring reliable real-time collision avoidance.

B. Related Works

1) *Safety-Critical Controller*: Safety-critical controllers that unify CLFs for stability and CBFs for safety through quadratic programming (CLF-CBF-QP) are proposed in [2], [3], and [4] to achieve adaptive cruise control (ACC). These controllers prioritize safety over other objectives like tracking, relaxing CLF constraints to satisfy the CBF constraints when they conflict. Furthermore, these safety-critical controllers have been applied to robotics, using CLFs for navigation and CBFs for safety guarantees [14], [15]. While many works focus on point-mass robot models with static obstacles, extensions to unicycle models are presented in [5] and [8]. However, a nominal CBF cannot manage both linear and angular velocities of the unicycle model to avoid collisions, as the CBF is designed based on the position coordinates of the robot without considering the robot's orientation, resulting in lacking control over steering [8]. To address this, Huang et al. [8] propose designing the CBF based on the center of the robot's rear axle

Received 11 December 2024; revised 6 February 2025; accepted 22 February 2025. Date of publication 7 March 2025; date of current version 20 August 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3301000, in part by the National Natural Science Foundation of China (NSFC) under Grant 62173297, in part by Zhejiang Key Research and Development Program under Grant 2022C01035, and in part by the Fundamental Research Funds for the Central Universities under Grant 226-2022-00086. Recommended by Associate Editor A. Mesbah. (*Corresponding author: Zhitao Liu.*)

Jihao Huang, Xuemin Chi, and Hongye Su are with the State Key Laboratory of Industrial Control Technology, Institute of Cyber-Systems and Control, Zhejiang University, Hangzhou 310027, China (e-mail: jihao@zju.edu.cn; chixuemin@zju.edu.cn; hysu@zju.edu.cn).

Jun Zeng and Koushil Sreenath are with the Hybrid Robotics Group, Department of Mechanical Engineering, University of California at Berkeley, Berkeley, CA 94720 USA (e-mail: zengjuns@berkeley.edu; koushils@berkeley.edu).

Zhitao Liu is with the Institute of Intelligence Science and Engineering, Shenzhen Polytechnic University, Shenzhen 518055, China (e-mail: ztliu@zju.edu.cn).

Digital Object Identifier 10.1109/TCST.2025.3546076

and its orientation, allowing the safety-critical controller to manage both linear and angular velocities to navigate the robot to its destination and avoid collisions with dynamic obstacles. As a result, it overcomes the limitation of traditional CBFs, which cannot account for steering to avoid collisions.

The extensions mentioned above mainly focus on velocity-controlled unicycle robot models, where the control inputs are linear and angular velocities. For acceleration-controlled unicycle robot models, safety-critical controllers are often designed using state-feedback-based CLFs and HOCBFs [11], [16], [17], [18]. The difficulty in designing CLFs and CBFs for high-order systems lies in the fact that CLFs are usually invalid and CBFs with a relative degree greater than one. The relative degree of a sufficiently differentiable function is defined as the number of times we need to differentiate it along the robot's dynamics until the control inputs explicitly appear [11]. Xiao et al. [18] propose two CLFs to achieve navigation: one state-feedback-based CLF to adjust the robot's orientation, and another to adjust its velocity toward the desired velocity. Moreover, since the relative degree of the CBF based on positions using Euclidean distance is two, Xiao and Belta [17] propose using HOCBF to generate valid collision avoidance constraints for the acceleration-controlled unicycle model, ensuring that the nominal CBF with relative degree two is non-negative. Since HOCBFs require all but the final derivative of the nominal CBF, where the control inputs explicitly appear, to be non-negative, the safety guarantee is only provided using a subset of the original safe set, leading to conservative control performance [19]. Moreover, computing proper constraint candidates with appropriate penalty weights and parameters for HOCBFs is costly and lacks geometrical intuition. Additionally, Ames et al. [20] propose the integral CBF (I-CBF), which guarantees safety in both states and inputs while minimally modifying a nominal dynamically defined controller. When the control inputs do not appear in the derivative of the original CBF, I-CBF also constructs a valid CBF following the concept of HOCBF. Consequently, it faces similar challenges as HOCBF. To address these issues and avoid the need for HOCBFs, we propose constructing CBFs in the velocity space for the acceleration-controlled unicycle models, offering an efficient and effective solution for dynamic collision avoidance. Moreover, unlike I-CBF [20], which relies on a dynamically defined tracking controller for navigation, our approach integrates state-feedback-based CLFs for navigation and CBFs for collision avoidance within a unified framework.

2) *Velocity Obstacle*: Collision cone (CC)-based methods [12], [21], [22], [23] are commonly used for local collision avoidance between circular robots and obstacles by defining a geometric conic set in the robot's velocity space. The CC considers the relative velocity between the robot and the obstacle, while VO is a specific type of CC that focuses on the robot's velocity. Additionally, VO-based methods have been extended to polytopic-shaped robots and obstacles [24]. Under the assumption that both the robot and the obstacle maintain their current velocities, if the robot's current velocity is within the VO induced by the obstacle, then it will collide with the obstacle at some future moment; conversely, obstacle

avoidance is guaranteed if the robot's current velocity is outside the VO. Therefore, the robot can avoid collisions with all obstacles in the environment by selecting a velocity outside any VO induced by all obstacles. Since VO explicitly considers the velocity of the obstacle and defines the set of unsafe velocities for the robot, some works [13], [25] use VO or its variants to formulate the collision avoidance constraints for acceleration-controlled robot models. Zhang et al. [13] formulate a constrained nonlinear model predictive control (NMPC) problem to achieve navigation and collision avoidance in distributed multirobot systems. This work employs the disjunction of two linear constraints to represent the safe velocity set of the robot, i.e., the complement set of the VO, with each linear constraint requiring that the robot's velocity lies within a half-space. However, the collision avoidance constraint that the robot's velocity should fall within the safe velocity set cannot be directly represented in terms of single constraint, so integer variables are introduced to ensure that at least one of the two linear constraints is satisfied, converting the problem into a mixed-integer nonlinear programming (MINLP) problem. When the prediction horizon is long, solving the MINLP problem becomes time-consuming, compromising real-time performance guarantees. To address this, we propose constructing VOCBFs based on the constraints in [13], transforming the optimization problem into a QP, thereby ensuring real-time performance. In addition, Thontepu et al. [19] propose constructing a CC-based CBF (C3BF) for the acceleration-controlled unicycle robot model, and real-time performance is ensured as the optimization problem is formulated as a QP. However, C3BF [19] is constructed based on the relative velocity between the robot and obstacles, and when applied to distributed multirobot systems, it neglects the reactive nature between robots, treating them as obstacles. In contrast, VOCBF is constructed based on the robot's velocity and allows for the incorporation of various VO variants which consider the reactive nature among robots, making it more suitable for distributed multirobot systems.

C. Contributions

In this article, we propose a safety-critical controller in the form of CLF-VOCBF-mixed-integer QP (MIQP) for the acceleration-controlled unicycle robot model. The key contributions of our work are as follows.

- 1) State-feedback-based CLFs are designed for navigation, with additional CLFs tailored for specific purposes.
- 2) VOCBFs are designed for dynamic collision avoidance and are further extended with VO variants for improved applicability in distributed multirobot systems.
- 3) The constraints of CLFs, VOCBFs, and physical capabilities, along with the objective function, are formulated into a CLF-VOCBF-MIQP. Furthermore, we propose splitting the MIQP into multiple suboptimization problems and employing a decision network to improve computational efficiency.
- 4) Extensive numerical simulations are conducted to validate the effectiveness of our proposed approach, and better dynamic obstacle avoidance performance is

demonstrated compared to HOCBF. Furthermore, successful navigation for distributed multirobot systems with guaranteed collision avoidance is achieved.

The rest of this article is organized as follows: In Section II, we formally define the robot model and the problem studied in this article, and review the concepts of CLF, CBF, and VO. The detailed design of the safety-critical controller is outlined in Section III, including the design of CLFs, VOCBFs, and the integrated controller. Numerical simulation results are presented in Section IV to demonstrate the effectiveness of our approach. Section V concludes the article.

II. PRELIMINARIES

Consider an affine control system in the form of

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u} \quad (1)$$

where $\mathbf{x} \in \mathcal{D} \subset \mathbb{R}^n$, $\mathbf{u} \in \mathcal{U} \subset \mathbb{R}^m$, $\Delta\mathbf{u} \in \Delta\mathcal{U} \subset \mathbb{R}^m$, and $\mathbf{f} : \mathcal{D} \rightarrow \mathbb{R}^n$ and $\mathbf{g} : \mathcal{D} \rightarrow \mathbb{R}^{n \times m}$ are locally Lipschitz continuous on $\mathcal{D}$. Moreover, $\mathcal{D} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{x}_{\min} \leq \mathbf{x} \leq \mathbf{x}_{\max}\}$, $\mathcal{U} = \{\mathbf{u} \in \mathbb{R}^m \mid \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}\}$, and $\Delta\mathcal{U} = \{\Delta\mathbf{u} \in \mathbb{R}^m \mid \Delta\mathbf{u}_{\min} \leq \Delta\mathbf{u} \leq \Delta\mathbf{u}_{\max}\}$ denote the closed constraint sets of admissible states, control inputs, and the changes in control inputs, respectively. Here $\mathbf{x}_{\min}$, $\mathbf{u}_{\min}$, $\Delta\mathbf{u}_{\min}$ and $\mathbf{x}_{\max}$, $\mathbf{u}_{\max}$, $\Delta\mathbf{u}_{\max}$ represent the lower and upper bounds of $\mathbf{x}$, $\mathbf{u}$, and $\Delta\mathbf{u}$, respectively.

A. Problem Formulation

Assume there is a set of N robots sharing an environment with a set of M static and dynamic obstacles, where both the robots and obstacles are circular-shaped. For notations, subscripts i and j are used to distinguish different robots and obstacles, where each robot and obstacle is represented by $R_i, i \in \{0, 1, \dots, N-1\}$ and $O_j, j \in \{0, 1, \dots, M-1\}$. Moreover, we mainly focus on robots with an acceleration-controlled unicycle model in this work. Since the circular-shaped robot has two axes, we model the acceleration-controlled unicycle model using the rear axle as follows:

$$\begin{bmatrix} \dot{x}_p \\ \dot{y}_p \\ \dot{\theta} \\ \dot{v} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ \alpha \end{bmatrix} \quad (2)$$

where x_p and y_p denote the current coordinates of the rear axle axis, and the center (x_c, y_c) of the circular-shaped robots can be represented in terms of (x_p, y_p) as follows:

$$\begin{bmatrix} x_c \\ y_c \end{bmatrix} = \begin{bmatrix} x_p + l \cos \theta \\ y_p + l \sin \theta \end{bmatrix} \quad (3)$$

where l represents the distance between the rear axle axis and the center, as shown in Fig. 1. Moreover, θ denotes the orientation with respect to the x -axis; v and ω denote the linear and angular velocities of the robot; and a and α denote the linear and angular accelerations that control the motion of the robot. Equation (2) is in the form of (1), and we have $\mathbf{x} = [x_p, y_p, \theta, v, \omega]^T \in \mathbb{R}^5$ and $\mathbf{u} = [a, \alpha]^T \in$

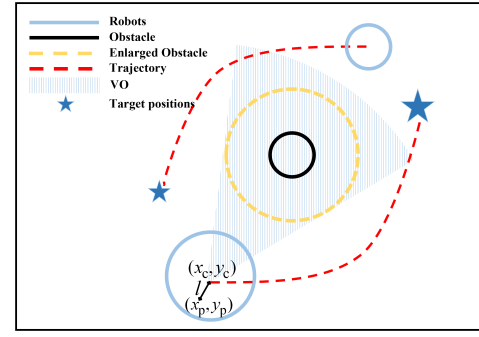


Fig. 1. Collision avoidance between the robot and obstacles is achieved using a VO-based approach, where the VO is depicted as the shaded area. The obstacle is enlarged by inflating it with the robot's radius, which is then used to construct the VO. Here, (x_p, y_p) denotes the coordinates of the rear axle axis, (x_c, y_c) represents the robot's center, and l is the distance between them.

$\mathbb{R}^2$. $\mathbf{x}_{\min}$, $\mathbf{u}_{\min}$, $\Delta\mathbf{u}_{\min}$, $\mathbf{x}_{\max}$, $\mathbf{u}_{\max}$, and $\Delta\mathbf{u}_{\max}$ represent the physical limits of states and control inputs, such as maximum velocity, acceleration, and the change rate of the acceleration. Additionally, the double-integrator dynamics model of the obstacle is defined as $[\dot{x}_o, \dot{y}_o, \ddot{x}_o, \ddot{y}_o]^T = [v_{ox}, v_{oy}, a_{ox}, a_{oy}]^T$, where x_o and y_o denote the horizontal and vertical positions of the obstacle; v_{ox} , v_{oy} , a_{ox} , and a_{oy} represent its horizontal and vertical velocities and accelerations. The states and control inputs of the obstacle are denoted as $\mathbf{x}_O = [x_o, y_o, v_{ox}, v_{oy}]^T$ and $\mathbf{u}_O = [a_{ox}, a_{oy}]^T$, respectively.

In our work, each robot operates as an individual and can make decisions independently based on the environmental information to reach its target position while avoiding collisions with other robots and obstacles, as shown in Fig. 1. The task of each robot is formulated as a constrained optimal control problem (COCP)

$$\begin{aligned} J(\mathbf{u}(t)) &= \min_{\mathbf{u}} \int_{t_0}^{t_f} \frac{1}{2} \mathbf{u}(t)^T \mathbf{H} \mathbf{u}(t) \\ \text{s.t. } \dot{\mathbf{x}}(t) &= \mathbf{f}(\mathbf{x}(t)) + \mathbf{g}(\mathbf{x}(t))\mathbf{u}(t) \\ \mathbf{x}(t) &\in \mathcal{D}, \quad \mathbf{x}(t) \in \mathcal{C}_{\text{Safe}} \\ \mathbf{u}(t) &\in \mathcal{U}, \quad \Delta\mathbf{u}(t) \in \Delta\mathcal{U} \end{aligned} \quad (4)$$

where $\mathbf{H}$ is a positive definite matrix, and $\mathcal{C}_{\text{Safe}}$ represents the safe region in the configure space, with t_0 and t_f denoting the initial and final times, respectively. The objective function of the optimization problem is to minimize energy consumption. Additionally, the COCP considers various constraints, including kinematic constraints such as the robot's dynamics, physical limits of states and control inputs, and collision avoidance constraints.

B. Control Lyapunov Function

Definition 1 (Class $\mathcal{K}$ and $\mathcal{K}_{\infty}$ Functions [26]): A Lipschitz continuous function $\mu : [0, a) \rightarrow [0, \infty)$, $a > 0$ is said to belong to class $\mathcal{K}$ if it is strictly increasing and satisfies $\mu(0) = 0$. Moreover, this function is said to belong to class $\mathcal{K}_{\infty}$ if it belongs to class $\mathcal{K}$ and further satisfies $a = \infty$ and $\mu(b) \rightarrow \infty$ as $b \rightarrow \infty$.

Definition 2 (Control Lyapunov Function [26]): A continuously differentiable function $V : \mathcal{D} \rightarrow \mathbb{R}$ is a CLF for

system (1) if it is positive definite and satisfies

$$\inf_{\mathbf{u} \in \mathcal{U}} [L_f V(\mathbf{x}) + L_g V(\mathbf{x}) \mathbf{u}] \leq -\gamma(V(\mathbf{x})) \quad (5)$$

where $L_f V(\mathbf{x}) := (\partial V / \partial \mathbf{x}) f(\mathbf{x})$ and $L_g V(\mathbf{x}) := (\partial V / \partial \mathbf{x}) g(\mathbf{x})$ are Lie derivatives of $V(\mathbf{x})$, and $\gamma(\cdot)$ belongs to class $\mathcal{K}$.

Theorem 1: Given a CLF $V(\mathbf{x})$ defined in Definition 2, any Lipschitz continuous controller $\mathbf{u} \in K_{\text{clf}}(\mathbf{x})$, with

$$K_{\text{clf}}(\mathbf{x}) := \{\mathbf{u} \in \mathcal{U}, L_f V(\mathbf{x}) + L_g V(\mathbf{x}) \mathbf{u} \leq -\gamma(V(\mathbf{x}))\} \quad (6)$$

can stabilize the system (1) to its desired states [26].

C. Control Barrier Function

Definition 3 (Forward Invariance Set): A set $\mathcal{C} \subset \mathcal{D}$ is forward invariant with respect to (1) if for every initial state $\mathbf{x}_0 \in \mathcal{C}$ and $\mathbf{x}(t) \in \mathcal{C}$, for $\mathbf{x}(t_0) = \mathbf{x}_0, \forall t \geq t_0$. The system (1) is safe with respect to the set $\mathcal{C}$ if $\mathcal{C}$ is forward invariant.

Compared to CLFs, which are proposed to stabilize the system (1) to an equilibrium point, CBFs are proposed to ensure the forward invariance of a set, as defined in Definition 3. Consider a set $\mathcal{C} \subset \mathcal{D}$ defined as a zero-superlevel set of a continuously differentiable function $h: \mathcal{D} \rightarrow \mathbb{R}$, yielding

$$\begin{aligned} \mathcal{C} &= \{\mathbf{x} \in \mathcal{D} \subset \mathbb{R}^n : h(\mathbf{x}) \geq 0\} \\ \partial \mathcal{C} &= \{\mathbf{x} \in \mathcal{D} \subset \mathbb{R}^n : h(\mathbf{x}) = 0\} \\ \text{Int}(\mathcal{C}) &= \{\mathbf{x} \in \mathcal{D} \subset \mathbb{R}^n : h(\mathbf{x}) > 0\}. \end{aligned} \quad (7)$$

Throughout this article, we refer to $\mathcal{C}$ as the safe set.

Definition 4 (Control Barrier Function [26]): Suppose the set $\mathcal{C}$ defined in (7) is the superlevel set of a continuously differentiable function $h: \mathcal{D} \rightarrow \mathbb{R}$, then h is a CBF if there exists an extended class $\mathcal{K}_\infty$ function $\mu(\cdot)$ such that for the control system (1)

$$\sup_{\mathbf{u} \in \mathcal{U}} [L_f h(\mathbf{x}) + L_g h(\mathbf{x}) \mathbf{u}] \geq -\mu(h(\mathbf{x})) \quad (8)$$

where $L_f h(\mathbf{x}) = ((\partial h(\mathbf{x})) / \partial \mathbf{x}) f(\mathbf{x})$ and $L_g h(\mathbf{x}) = ((\partial h(\mathbf{x})) / \partial \mathbf{x}) g(\mathbf{x})$ are Lie derivatives of $h(\mathbf{x})$.

Theorem 2: If $h(\mathbf{x})$ is a CBF on $\mathcal{C}$ and $((\partial h(\mathbf{x})) / \partial \mathbf{x}) \neq 0, \forall \mathbf{x} \in \partial \mathcal{C}$, then any Lipschitz continuous controller $\mathbf{u} \in K_{\text{cbf}}(\mathbf{x})$, with

$$K_{\text{cbf}}(\mathbf{x}) := \{\mathbf{u} \in \mathcal{U}, L_f h(\mathbf{x}) + L_g h(\mathbf{x}) \mathbf{u} \geq -\mu(h(\mathbf{x}))\} \quad (9)$$

can ensure the forward invariance of $\mathcal{C}$ and thus the safety of system (1) [26].

Since constraints (6) and (9) are affine in the control inputs $\mathbf{u}$, many studies [2], [3] propose a safety-critical controller which unifies CLF-CBF-QP, formulated as

CLF-CBF-QP:

$$\min_{(\mathbf{u}, \delta) \in \mathbb{R}^{m+1}} \frac{1}{2} \mathbf{u}^\top H \mathbf{u} + p \delta^2 \quad (10a)$$

$$\text{s.t. } L_f V(\mathbf{x}) + L_g V(\mathbf{x}) \mathbf{u} + \gamma(V(\mathbf{x})) \leq \delta \quad (10b)$$

$$L_f h(\mathbf{x}) + L_g h(\mathbf{x}) \mathbf{u} \geq -\mu(h(\mathbf{x})) \quad (10c)$$

$$\mathbf{u} \in \mathcal{U}, \Delta \mathbf{u} \in \Delta \mathcal{U} \quad (10d)$$

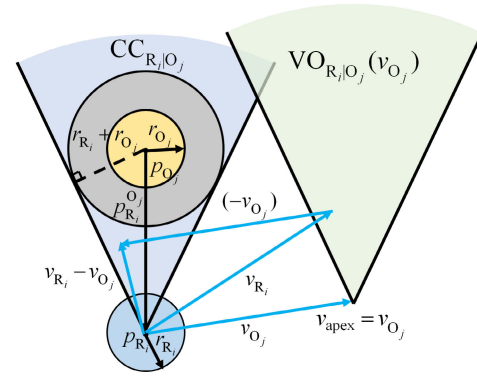


Fig. 2. Velocity obstacle $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ of robot R_i induced by the obstacle O_j , and $\text{CC}_{R_i|O_j}$ represents the CC between them. If the relative velocity $\mathbf{v}_{R_i} - \mathbf{v}_{O_j} \in \text{CC}_{R_i|O_j}$ or the robot's velocity $\mathbf{v}_{R_i} \in \text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$, then a collision will occur between R_i and O_j at some future time with the assumption that both maintain their current velocities.

where H is a predefined positive definite matrix, and $p > 0$ is a weighting factor to minimize δ , where δ is a relaxation variable to relax the CLF constraint (10b) to ensure safety when the CLF constraint conflicts with the CBF constraint (10c). The objective function (10a) aims to minimize energy consumption and reduce the additional quadratic cost associated with the relaxation variable.

The original COCP (4) can be reformulated as a sequence of QPs in the form of (10), i.e., the time interval $[t_0, t_f]$ is partitioned into a set of equal time intervals $\{[t_0, t_0 + \Delta t), [t_0 + \Delta t, t_0 + 2\Delta t), \dots\}$, where $\Delta t > 0$. Within each interval $[t_0 + q\Delta t, t_0 + (q+1)\Delta t)$, for $q = 0, 1, 2, \dots$, the state maintains its value at the beginning of the interval, and the control inputs are obtained by solving (10). The state is then updated based on the applied control input, and the procedure is repeated for subsequent intervals. Therefore, this QP-based approach (10) is suboptimal compared to the original COCP (4), since the optimizations are performed pointwise in time.

D. Velocity Obstacle

Assume there is a robot R_i and an obstacle O_j with radii r_{R_i} and r_{O_j} . Their positions and velocities are denoted as $\mathbf{p}_{R_i}$, $\mathbf{p}_{O_j}$, $\mathbf{v}_{R_i}$, and $\mathbf{v}_{O_j}$, as shown in Fig. 2. The VO of R_i induced by O_j is denoted as $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$, which contains all velocities for R_i that would result in collisions with the obstacle in the future, assuming both maintain their current constant velocities. Let $A \oplus B = \{a + b | a \in A, b \in B\}$ be the Minkowski sum of sets A and B , and let $\lambda(\mathbf{p}, \mathbf{v}) = \{\mathbf{p} + t\mathbf{v} | t > 0\}$ denote a ray starting at position $\mathbf{p}$ and in the direction of vector $\mathbf{v}$. If a ray starting at $\mathbf{p}_{R_i}$ and heading in the direction of the relative velocity $\mathbf{v}_{R_i} - \mathbf{v}_{O_j}$ intersects the Minkowski sum of $O_j \oplus -R_i$ centered at $\mathbf{p}_{O_j}$, then $\mathbf{v}_{R_i}$ is in the VO of R_i induced by O_j . Hence, $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ is defined as follows [12]:

$$\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j}) = \{\mathbf{v}_{R_i} | \lambda(\mathbf{p}_{R_i}, \mathbf{v}_{R_i} - \mathbf{v}_{O_j}) \cap O_j \oplus -R_i \neq \emptyset\}.$$

Therefore, under the assumption that both R_i and O_j maintain their current constant velocities, if $\mathbf{v}_{R_i} \in \text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$, then a collision will occur between R_i and O_j at some future moment. Conversely, collision avoidance is achieved if $\mathbf{v}_{R_i} \notin \text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$. In fact, $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ is a velocity cone with its

apex at $\mathbf{v}_{O_j}$, as shown in Fig. 2. It can be transformed from the CC defined as

$$\text{CC}_{R_i|O_j} = \{\mathbf{v}_{R_i} - \mathbf{v}_{O_j} | \lambda(\mathbf{p}_{R_i}, \mathbf{v}_{R_i} - \mathbf{v}_{O_j}) \cap O_j \oplus -R_i \neq \emptyset\}$$

with the transformation $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j}) = \text{CC}_{R_i|O_j} \oplus \mathbf{v}_{O_j}$. The difference between these two cones lies in the position of their apexes: $\text{CC}_{R_i|O_j}$ has its apex at the origin since it considers the relative velocity, while $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ has its apex at $\mathbf{v}_{O_j}$.

VO is widely used to enable a robot to avoid collisions with multiple obstacles by having the robot select a velocity which lies outside any VO induced by each obstacle [12]. Several approaches have also been proposed to overcome the limitations of VO in avoiding collisions between robots, such as reciprocal VO (RVO) [21], hybrid RVO (HRVO) [22], and optimal reciprocal collision avoidance (ORCA) [23]. These methods reduce unnecessary oscillations by explicitly considering the reactive nature among robots. The challenges of VO lie in its reliance on the assumption that the velocity of the obstacle remains constant and its inability to explicitly consider the robot's physical limitation constraints on velocity and acceleration. In this work, we fully leverage the dynamic obstacle avoidance capabilities of VO to construct VO-based CBFs in the velocity space, instead of constructing CBF based on positions and introducing HOCBF for the acceleration-controlled unicycle model. For more details, readers can refer to Section III-B.

III. CONTROLLER DESIGN

In this section, we first present the design of state-feedback-based CLFs for the acceleration-controlled unicycle model (2) to achieve navigation. Next, we demonstrate the construction of VOCBFs in the velocity space to ensure dynamic collision avoidance. Additionally, we impose CBF constraints to account for the physical limitations of states. Finally, we demonstrate how to formulate the constraints of CLFs and VOCBFs, along with other physical limitation constraints on states and control inputs in the form of CLF-VOCBF-MIQP and describe how it can be efficiently solved using a decision network. For brevity, we omit the subscript i of robots and reserve i for other purposes.

A. Design of CLFs

CLFs are commonly used to stabilize states to their desired equilibrium points. When applying CLFs to the field of robotics, it is common to leverage their stabilizing properties for navigation purposes. In this section, we mainly demonstrate the design of state-feedback-based CLFs [27] to achieve navigation for the acceleration-controlled unicycle model (2). Specifically, we design four CLFs, each serving a different purpose: reducing the distance between the robot and its target position, adjusting the robot's orientation to ensure it moves toward the target, and enabling efficient and smooth motion.

Assume the robot's goal state is denoted as $(x_{gr}, y_{gr}, \theta_g)$, where (x_{gr}, y_{gr}) represents the coordinates of the rear axle axis. Additionally, the center position (x_g, y_g) of the goal state is defined as $x_g = x_{gr} + l \cos \theta_g$ and $y_g = y_{gr} + l \sin \theta_g$. Our primary objective is to stabilize the robot's position to its target

position (x_g, y_g) without explicitly considering other states such as v and ω in the dynamics (2). If we design a CLF such as $V(\mathbf{x}) = (x_c - x_g)^2 + (y_c - y_g)^2$, then $\forall \mathbf{x}, L_g V(\mathbf{x}) = 0$. This results in a high relative degree issue [11], preventing us from obtaining a valid controller. To address this problem, we adopt state-feedback-based CLFs [27], which mitigate the relative degree issue and facilitate controller design.

Remark 1: State-feedback-based CLFs stabilize a state to its desired state by making its higher order derivative inversely dependent on all lower order derivatives, ensuring all lower order derivatives eventually converge to their desired states. This method is especially effective for high-order systems.

Suppose $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top \in \mathbb{R}^n$, and we aim to stabilize only the partial states $[x_1, x_2, \dots, x_s]^\top \in \mathbb{R}^s, s < n$ to their desired states $[x_1^*, x_2^*, \dots, x_s^*]^\top$, with $\mathbf{u} \in \mathbb{R}^m$. Assume $\forall x_i, i \in \{1, 2, \dots, s\}$, control inputs $\mathbf{u}$ can influence the dynamics. Furthermore, the relative degree of $x_i, i \in \{1, 2, \dots, s\}$ is defined as $r_i \in \mathbb{N}$, and $n_0 := \sum_{i=1}^s r_i$ represents the sum of the relative degree. We recursively define new states by taking derivatives of x_i : the first derivative is $x_{i,1} \in \mathbb{R}$, the second derivative is $x_{i,2} \in \mathbb{R}$, and so on, until reaching $x_{i,r_i-1} \in \mathbb{R}$, where the first derivative of x_{i,r_i-1} becomes an affine function of $\mathbf{u}$, i.e., the relative degree of x_{i,r_i-1} is 1. In this work, we extend the concept of state-feedback-based CLFs, which were originally designed to stabilize x_i to the origin, to stabilize x_i to its desired state x_i^* . The desired state of x_{i,r_i-1} is defined as $x_{i,r_i-1}^* := -l_i(x_i - x_i^*) - l_{i,1}x_{i,1} - \dots - l_{i,r_i-2}x_{i,r_i-2}$, where $l_i > 0, l_{i,1} > 0, \dots, l_{i,r_i-2} > 0$. Since the desired state of x_{i,r_i-1} depends inversely on all lower relative degree states $x_{i,k}, k \in \{0, 1, \dots, r_i - 2\}$, stabilizing x_{i,r_i-1} to its desired state x_{i,r_i-1}^* will indirectly stabilize all lower relative degree states to their desired states, including x_i . Thus, we can design a CLF to stabilize x_{i,r_i-1} to its desired state x_{i,r_i-1}^* , which in turn stabilizes x_i to its desired states. This is referred to as a state-feedback-based CLF [27]. To integrate this design, we define an integrated state $\mathbf{y} := [x_1, \dots, x_s, x_{1,1}, \dots, x_{1,r_1-1}, \dots, x_{s,1}, \dots, x_{s,r_s-1}]^\top \in \mathbb{R}^{n_0}$ with its desired state $\mathbf{y}^* := [x_1^*, \dots, x_s^*, 0, \dots, 0]^\top$. The valid state-feedback-based CLF with relative degree 1 is defined as follows:

$$\begin{aligned} V(\mathbf{y}, \mathbf{y}^*) &= \sum_{i=1}^s c_i \left(\frac{1}{l_i} x_{i,r_i-1} - \frac{1}{l_i} x_{i,r_i-1}^* \right)^2 \\ &= \sum_{i=1}^s c_i (x_i - x_i^* + k_{i,1}x_{i,1} + \dots + k_{i,r_i-1}x_{i,r_i-1})^2 \end{aligned} \quad (11)$$

where $c_i \geq 0, i \in \{1, \dots, s\}$, and $k_{i,1} = (l_{i,1}/l_i), \dots, k_{i,r_i-2} = (l_{i,r_i-2}/l_i), k_{i,r_i-1} = (1/l_i) > 0$. With this state-feedback-based CLF, we can stabilize the states $[x_1, \dots, x_s]^\top$ to their desired states $[x_1^*, \dots, x_s^*]^\top$. Additionally, (11) can be rewritten in matrix form as follows:

$$V(\mathbf{y}, \mathbf{y}^*) = (\mathbf{Q}(\mathbf{y} - \mathbf{y}^*))^\top \Lambda (\mathbf{Q}(\mathbf{y} - \mathbf{y}^*)) \quad (12)$$

where

$$\mathbf{Q} = \begin{bmatrix} \mathbf{I}_{s \times s} & \mathbf{K} \\ \mathbf{0}_{(n_0-s) \times s} & \mathbf{0}_{(n_0-s) \times (n_0-s)} \end{bmatrix} \in \mathbb{R}^{n_0 \times n_0}$$

$$\mathcal{K} = \begin{bmatrix} \mathbf{k}_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{k}_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{k}_s \end{bmatrix} \in \mathbb{R}^{s \times (n_0 - s)}$$

$$\Lambda = \begin{bmatrix} \mathbf{c}_{s \times s} & \mathbf{0}_{s \times (n_0 - s)} \\ \mathbf{0}_{(n_0 - s) \times s} & \mathbf{0}_{(n_0 - s) \times (n_0 - s)} \end{bmatrix} \in \mathbb{R}^{n_0 \times n_0}$$

where $\mathcal{K}$ is a block diagonal matrix composed of vectors $\mathbf{k}_i = [k_{i,1}, \dots, k_{i,r_i-1}]$, $i \in \{1, \dots, s\}$, $\mathbf{c}_{s \times s}$ is a diagonal matrix composed of c_i , $i \in \{1, \dots, s\}$, $\mathbf{0}_{s \times (n_0 - s)}$ is a zero matrix of size $s \times (n_0 - s)$, and $\mathbf{I}_{s \times s}$ is an identity matrix of size $s \times s$. In addition, the CLF constraint (10b) can be derived for the state-feedback-based CLF [27].

To achieve navigation for the acceleration-controlled unicycle model (2), we design four distinct CLFs, each serving a specific purpose. First, we design a valid state-feedback-based CLF to reduce the distance between the robot's current position and its target position as follows:

$$V_d(\mathbf{x}) = c_1[x_c - x_g + k_1(v \cos \theta - l \sin \theta \omega)]^2 + c_2[y_c - y_g + k_2(v \sin \theta + l \cos \theta \omega)]^2 \quad (13)$$

where x_c and y_c denote the center position of the robot, as defined in (3). This CLF allows the controller to effectively manage both linear and angular accelerations, guiding the robot to its destination.

Remark 2: If (x_p, y_p) represents the robot's center position instead of the coordinates of the rear axle axis, the state-feedback-based CLF becomes $V_d(\mathbf{x}) = c_1(x_p - x_g + k_1 v \cos \theta)^2 + c_2(y_p - y_g + k_2 v \sin \theta)^2$. Additionally, $L_g V_d(\mathbf{x}) = [*, 0]$ in this case, meaning the controller's performance is limited since it cannot control the angular acceleration α . To address this limitation, we propose controlling (x_p, y_p) directly instead of (x_c, y_c) .

Additionally, to ensure the robot always moves toward its destination, we design another state-feedback-based CLF for the robot's orientation. Since the relative degree of $\theta - \arctan((y_g - y_c)/(x_g - x_c))$ is 2, the CLF is expressed as follows:

$$V_\theta(\mathbf{x}) = \left(\theta - \arctan\left(\frac{y_g - y_c}{x_g - x_c}\right) + k_\theta q_\theta \right)^2 \quad (14)$$

where q_θ is the 1st derivative of $\theta - \arctan((y_g - y_c)/(x_g - x_c))$. With this design, the relative degree of $V_\theta(\mathbf{x})$ is 1, ensuring control inputs explicitly appear in the constraints of $V_\theta(\mathbf{x})$.

Besides, we design a CLF that enables the robot to adjust its velocity to approach its desired velocity. When far from the target, the robot moves faster to reduce travel time; as it nears the target, it slows down. The CLF is defined as follows:

$$V_v(\mathbf{x}) = (v - v_d)^2 \quad (15)$$

where the desired velocity v_d is proportional to the distance between the robot's current position and its target. Moreover, to ensure smooth motion, we design a CLF as follows:

$$V_\omega(\mathbf{x}) = \omega^2. \quad (16)$$

Since $L_g V_v(\mathbf{x}) = [2(v - v_d), 0]$ and $L_g V_\omega(\mathbf{x}) = [0, 2\omega]$, both CLFs are valid without requiring state-feedback-based designs.

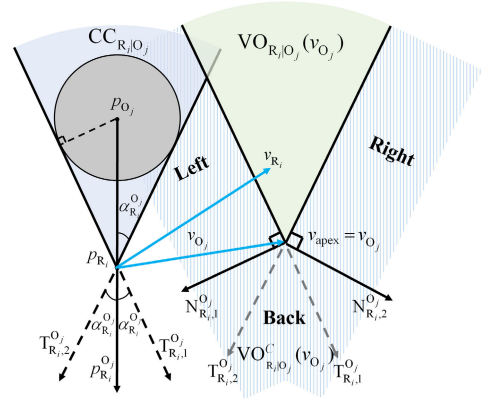


Fig. 3. Safe range of the robot's velocity is denoted by $\text{VO}_{R_i|O_j}^C(v_{O_j})$, and it can be represented by the disjunction of two linear constraints, where each linear constraint requires that the robot's velocity is within a half-space. Each half-space corresponds to a direction in which the robot can navigate around the obstacle, and satisfying both leads to backward avoidance.

In conclusion, we design four CLFs $V_d(\mathbf{x})$, $V_\theta(\mathbf{x})$, $V_v(\mathbf{x})$, and $V_\omega(\mathbf{x})$ to achieve navigation. Among these, $V_d(\mathbf{x})$ and $V_\theta(\mathbf{x})$ serve as the primary CLFs to guide the robot to its destination, while $V_v(\mathbf{x})$ and $V_\omega(\mathbf{x})$ play auxiliary roles. This prioritization is managed by adjusting the weight factors p of the relaxation variables in (10a) for each CLF constraint. Specifically, a small weight factor increases the level of relaxation, making the CLF auxiliary. A large weight factor reduces relaxation, giving the CLF a primary role.

B. Design of CBFs

In this section, we mainly demonstrate how to construct VOCBFs for the acceleration-controlled unicycle model (2) to achieve dynamic collision avoidance. Consider a CBF constructed based on the Euclidean distance between robot R_i and obstacle O_j , defined as follows:

$$h(\mathbf{x}) = \|\mathbf{p}_{R_i} - \mathbf{p}_{O_j}\|^2 - (r_{R_i} + r_{O_j})^2. \quad (17)$$

Since the relative degree of (17) is 2 with respect to the robot's dynamics (2), HOCBF is necessary to be introduced to generate valid collision avoidance constraints with the safety guarantee that (17) is always non-negative [11]. However, HOCBF only provides a subset of the original safe set for safety guarantees, since it requires all but the final derivative of the nominal CBF, where the control inputs explicitly appear, to be non-negative, which can lead to the optimization problem being conservative or even infeasible in certain cases [19]. Additionally, the calculation of proper constraint candidates of HOCBF with appropriate penalty weights and parameters is computationally expensive [17]. To overcome these limitations and design a valid CBF with a relative degree of 1, we propose constructing VOCBFs directly in the velocity space, eliminating the need for HOCBFs.

As mentioned in Section II-D, the VO of R_i induced by O_j is denoted as $\text{VO}_{R_i|O_j}(v_{O_j})$. Under the assumption that both R_i and O_j maintain their current velocities, R_i will collide with O_j at some future moment if $\mathbf{v}_{R_i} \in \text{VO}_{R_i|O_j}(v_{O_j})$. Conversely, collision avoidance is guaranteed if $\mathbf{v}_{R_i} \notin \text{VO}_{R_i|O_j}(v_{O_j})$, i.e., $\mathbf{v}_{R_i} \in \text{VO}_{R_i|O_j}^C(v_{O_j})$. Here, $\text{VO}_{R_i|O_j}^C(v_{O_j})$ is the complement of $\text{VO}_{R_i|O_j}(v_{O_j})$ and represents the feasible velocity region

for $\mathbf{v}_{R_i}$, as shown in the blue shaded area of Fig. 3. Let $\mathbf{p}_{R_i}^{O_j} := \mathbf{p}_{R_i} - \mathbf{p}_{O_j}$ and $\mathbf{v}_{R_i}^{O_j} := \mathbf{v}_{R_i} - \mathbf{v}_{O_j}$ represent the relative position and velocity between R_i and O_j , respectively. The angle between $\mathbf{p}_{R_i}^{O_j}$ and the boundary line of $\text{CC}_{R_i|O_j}$ is defined as $\alpha_{R_i}^{O_j}$, where $\alpha_{R_i}^{O_j} \in (0, (\pi/2))$ and $\sin \alpha_{R_i}^{O_j} = ((r_{R_i} + r_{O_j}) / \|\mathbf{p}_{R_i}^{O_j}\|)$, as shown in Fig. 3. The two tangent directional vectors of $\text{CC}_{R_i|O_j}$, denoted as $\mathbf{T}_{R_i,1}^{O_j}$ and $\mathbf{T}_{R_i,2}^{O_j}$, can be obtained by rotating the vector $\mathbf{p}_{R_i}^{O_j}$ counterclockwise and clockwise by the angle $\alpha_{R_i}^{O_j}$, respectively,

$$\mathbf{T}_{R_i,1}^{O_j} = \mathbf{R}(\alpha_{R_i}^{O_j}) \mathbf{p}_{R_i}^{O_j}, \quad \mathbf{T}_{R_i,2}^{O_j} = \mathbf{R}(-\alpha_{R_i}^{O_j}) \mathbf{p}_{R_i}^{O_j} \quad (18)$$

where

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

represents the rotation matrix that rotates a vector counterclockwise by the angle θ . Additionally, two outer normal vectors $\mathbf{N}_{R_i,1}^{O_j}$ and $\mathbf{N}_{R_i,2}^{O_j}$ can be obtained by rotating $\mathbf{T}_{R_i,1}^{O_j}$ clockwise with the angle $(\pi/2)$ and rotating $\mathbf{T}_{R_i,2}^{O_j}$ counterclockwise with the angle $(\pi/2)$ as follows:

$$\mathbf{N}_{R_i,1}^{O_j} = \mathbf{R}(-\frac{\pi}{2}) \mathbf{T}_{R_i,1}^{O_j}, \quad \mathbf{N}_{R_i,2}^{O_j} = \mathbf{R}(\frac{\pi}{2}) \mathbf{T}_{R_i,2}^{O_j}. \quad (19)$$

The outer normal vectors and the tangent directional vectors of $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ are identical to those of $\text{CC}_{R_i|O_j}$, since $\text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ is translated from $\text{CC}_{R_i|O_j}$. Given that collision avoidance between R_i and O_j is ensured if $\mathbf{v}_{R_i} \notin \text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$, i.e., $\mathbf{v}_{R_i} \in \text{VO}_{R_i|O_j}^C(\mathbf{v}_{O_j})$. This condition can be represented by the disjunction of two linear constraints as follows:

$$\mathbf{v}_{R_i} \in \text{VO}_{R_i|O_j}^C(\mathbf{v}_{O_j}) \iff \bigcup_{k \in \{1,2\}} (\mathbf{N}_{R_i,k}^{O_j})^\top \mathbf{v}_{R_i} \geq c_{R_i,k}^{O_j} \quad (20)$$

where $c_{R_i,k}^{O_j}$, $k \in \{1,2\}$ is the scalar corresponding to the k th linear constraint of $\text{VO}_{R_i|O_j}^C(\mathbf{v}_{O_j})$, given by

$$c_{R_i,1}^{O_j} = (\mathbf{N}_{R_i,1}^{O_j})^\top \mathbf{v}_{O_j}, \quad c_{R_i,2}^{O_j} = (\mathbf{N}_{R_i,2}^{O_j})^\top \mathbf{v}_{O_j}.$$

As indicated by (20), $\text{VO}_{R_i|O_j}^C(\mathbf{v}_{O_j})$ is represented by the combination of two half-spaces. In fact, each half-space corresponds to a direction in which the robot can navigate around the obstacle. If $\mathbf{v}_{R_i}$ only satisfies $(\mathbf{N}_{R_i,1}^{O_j})^\top \mathbf{v}_{R_i} \geq c_{R_i,1}^{O_j}$, then the robot will navigate around the obstacle from its left side. Here, “left” specifically refers to the left side relative to the robot’s local coordinate frame, as shown in Fig. 3. Conversely, if $\mathbf{v}_{R_i}$ only satisfies $(\mathbf{N}_{R_i,2}^{O_j})^\top \mathbf{v}_{R_i} \geq c_{R_i,2}^{O_j}$, then the robot will pass the obstacle from the right side. If $\mathbf{v}_{R_i}$ satisfies both $(\mathbf{N}_{R_i,1}^{O_j})^\top \mathbf{v}_{R_i} \geq c_{R_i,1}^{O_j}$ and $(\mathbf{N}_{R_i,2}^{O_j})^\top \mathbf{v}_{R_i} \geq c_{R_i,2}^{O_j}$, then the robot will move backward to avoid the obstacle. Furthermore, (20) can be reformulated using the relative velocity $\mathbf{v}_{R_i}^{O_j}$ as follows:

$$\bigcup_{k \in \{1,2\}} (\mathbf{N}_{R_i,k}^{O_j})^\top \mathbf{v}_{R_i}^{O_j} \geq 0. \quad (21)$$

If the robot’s velocity satisfies (21), then collision avoidance is guaranteed between R_i and O_j . We construct two VOCBFs based on these two linear constraints (21) as follows:

$$h_{R_i,1}^{O_j}(\mathbf{x}) = (\mathbf{N}_{R_i,1}^{O_j})^\top \mathbf{v}_{R_i}^{O_j} = (\mathbf{v}_{R_i}^{O_j})^\top \mathbf{N}_{R_i,1}^{O_j} \quad (22)$$

$$h_{R_i,2}^{O_j}(\mathbf{x}) = (\mathbf{N}_{R_i,2}^{O_j})^\top \mathbf{v}_{R_i}^{O_j} = (\mathbf{v}_{R_i}^{O_j})^\top \mathbf{N}_{R_i,2}^{O_j}. \quad (23)$$

The relative position and velocity between R_i and O_j are computed as follows:

$$\mathbf{p}_{R_i}^{O_j} = \begin{bmatrix} x_{p,i} + l_i \cos \theta_i - x_{o,j} \\ y_{p,i} + l_i \sin \theta_i - y_{o,j} \end{bmatrix} = \begin{bmatrix} \Delta x_i^j \\ \Delta y_i^j \end{bmatrix}$$

$$\mathbf{v}_{R_i}^{O_j} = \frac{d\mathbf{p}_{R_i}^{O_j}}{dt} = \begin{bmatrix} v_i \cos \theta_i - l_i \sin \theta_i \omega_i - v_{ox,j} \\ v_i \sin \theta_i + l_i \cos \theta_i \omega_i - v_{oy,j} \end{bmatrix}$$

where center positions of R_i and O_j are given by $\mathbf{p}_{R_i} = [x_{p,i} + l_i \cos \theta_i, y_{p,i} + l_i \sin \theta_i]^\top$ and $\mathbf{p}_{O_j} = [x_{o,j}, y_{o,j}]^\top$, respectively. Using (18) and (19), the VOCBFs (22) and (23) can be expressed as follows:

$$h_{R_i,1}^{O_j}(\mathbf{x}) = (\mathbf{v}_{R_i}^{O_j})^\top \mathbf{R}(\alpha_{R_i}^{O_j} - \frac{\pi}{2}) \mathbf{p}_{R_i}^{O_j} \quad (24)$$

$$h_{R_i,2}^{O_j}(\mathbf{x}) = (\mathbf{v}_{R_i}^{O_j})^\top \mathbf{R}(\frac{\pi}{2} - \alpha_{R_i}^{O_j}) \mathbf{p}_{R_i}^{O_j}. \quad (25)$$

It is important to note that for collision avoidance, at least one of these two VOCBFs (22) and (23) must be non-negative. Therefore, at least one of the two corresponding CBF constraints (10c) must be satisfied. In fact, satisfying only one of these two constraints is sufficient in most case, meaning the robot can simply choose a direction to navigate around the obstacle.

Assumption 1: $\alpha_{R_i}^{O_j} \neq (\pi/2)$, i.e., $\cos \alpha_{R_i}^{O_j} \neq 0$.

Assumption 2: $\cos \theta_i \neq 0$.

Remark 3: Assumption 1 may become invalid when the distance $\|\mathbf{p}_{R_i}^{O_j}\|$ between the robot and the obstacle equals $r_{R_i} + r_{O_j}$. Similarly, Assumption 2 could be invalid when the robot’s orientation equals $(\pi/2)$ or $-(\pi/2)$. Without loss of generality, we assume both Assumptions 1 and 2 hold true.

Theorem 3: Given the acceleration-controlled model (2), the proposed VOCBF candidates are valid CBFs under Assumptions 1 and 2, i.e., $\forall \mathbf{x} \in \mathcal{D}, L_g h_{R_i,1}^{O_j}(\mathbf{x}) \neq 0, L_g h_{R_i,2}^{O_j}(\mathbf{x}) \neq 0$.

Proof: We demonstrate the validity of the VOCBF constraints using (24) as an example, as the proof of (25) follows a similar approach. The time derivative of $h_{R_i,1}^{O_j}(\mathbf{x})$ is given by

$$\frac{dh_{R_i,1}^{O_j}(\mathbf{x})}{dt} = \left(\frac{d\mathbf{v}_{R_i}^{O_j}}{dt} \right)^\top \mathbf{R}(\alpha_{R_i}^{O_j} - \frac{\pi}{2}) \mathbf{p}_{R_i}^{O_j} \quad (26)$$

$$+ (\mathbf{v}_{R_i}^{O_j})^\top \frac{d\mathbf{R}(\alpha_{R_i}^{O_j} - \frac{\pi}{2})}{dt} \mathbf{p}_{R_i}^{O_j} \quad (27)$$

$$+ (\mathbf{v}_{R_i}^{O_j})^\top \mathbf{R}(\alpha_{R_i}^{O_j} - \frac{\pi}{2}) \frac{d\mathbf{p}_{R_i}^{O_j}}{dt} \quad (28)$$

where the derivative can also be expressed as

$$\frac{dh_{R_i,1}^{O_j}(\mathbf{x})}{dt} = L_f h_{R_i,1}^{O_j}(\mathbf{x}) + L_g h_{R_i,1}^{O_j}(\mathbf{x}) \mathbf{u} + \frac{dh_{R_i,1}^{O_j}(\mathbf{x})}{d\mathbf{x}_{O_j}} \frac{d\mathbf{x}_{O_j}}{dt} \quad (29)$$

where we consider time-varying CBFs [8] to handle dynamic obstacles. Only the term (26) is related to the control inputs $\mathbf{a}$ and α , while other terms (27) and (28) do not contain any

terms related to a and α . Extracting the term related to $u = [a, \alpha]^\top$ from (26), we derive

$$L_g h_{R_i,1}^{O_j}(x) = \left[\left\langle R(\alpha_{R_i}^{O_j} - \frac{\pi}{2}) p_{R_i}^{O_j}, \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix} \right\rangle \right. \\ \left. \left\langle R(\alpha_{R_i}^{O_j} - \frac{\pi}{2}) p_{R_i}^{O_j}, \begin{bmatrix} -l \sin \theta_i \\ l \cos \theta_i \end{bmatrix} \right\rangle \right]^\top$$

where $\langle a, b \rangle$ represents the inner product between a and b .

To prove that $L_g h_{R_i,1}^{O_j}(x) \neq 0, \forall x \in \mathcal{D}$, we proceed by contradiction. Suppose $L_g h_{R_i,1}^{O_j}(x) = 0$, this leads to the following possible scenarios.

- 1) *Scenario 1:* $R(\alpha_{R_i}^{O_j} - (\pi/2)) p_{R_i}^{O_j} = 0$, which implies $L_g h_{R_i,1}^{O_j}(x) \equiv 0$. This further implies $\sin \alpha_{R_i}^{O_j} \Delta x_i^j + \cos \alpha_{R_i}^{O_j} \Delta y_i^j = 0$ and $-\cos \alpha_{R_i}^{O_j} \Delta x_i^j + \sin \alpha_{R_i}^{O_j} \Delta y_i^j = 0$. Under Assumption 1, this results in $\Delta x_i^j = \Delta y_i^j = 0$, implying that the centers of R_i and O_j coincide. This is impossible, as it would mean the robot is already inside the obstacle.
- 2) *Scenario 2:* $R(\alpha_{R_i}^{O_j} - (\pi/2)) p_{R_i}^{O_j} \perp \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix}$ and $R(\alpha_{R_i}^{O_j} - (\pi/2)) p_{R_i}^{O_j} \perp \begin{bmatrix} -l \sin \theta_i \\ l \cos \theta_i \end{bmatrix}$. Let $R(\alpha_{R_i}^{O_j} - (\pi/2)) p_{R_i}^{O_j} = [s_1, s_2]^\top$, then $s_1 \cos \theta_i + s_2 \sin \theta_i = 0$ and $s_1 \sin \theta_i = -s_2 \cos \theta_i$. Under Assumption 2, we derive $s_1 = s_2 = 0$, returning to Scenario 1, which is also impossible.

Thus, $L_g h_{R_i,1}^{O_j}(x) \neq 0$, indicating that the constraint of $h_{R_i,1}^{O_j}(x)$ is always valid. Similarly, for (25), we derive

$$L_g h_{R_i,2}^{O_j}(x) = \left[\left\langle R(\frac{\pi}{2} - \alpha_{R_i}^{O_j}) p_{R_i}^{O_j}, \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix} \right\rangle \right. \\ \left. \left\langle R(\frac{\pi}{2} - \alpha_{R_i}^{O_j}) p_{R_i}^{O_j}, \begin{bmatrix} -l \sin \theta_i \\ l \cos \theta_i \end{bmatrix} \right\rangle \right]^\top.$$

The proof that $L_g h_{R_i,2}^{O_j}(x) \neq 0$ is analogous and is omitted for brevity. ■

We have introduced how to construct VOCBFs based on $VO_{R_i|O_j}(v_{O_j})$ to achieve collision avoidance between R_i and O_j . Similarly, to avoid collisions between robots R_i and R_j , VOCBFs can also be constructed based on $VO_{R_i|R_j}(v_{R_j})$. However, since VO ignores the reactive nature between robots, which enables each robot to independently adjust its velocity to avoid collision with other robots and obstacles, using VO for navigation and collision avoidance in distributed multirobot systems may bring unnecessary oscillations. Some variants of VO like RVO [21] and HRVO [22] explicitly consider the reactive nature of robots, leading to improved performance without unnecessary oscillations. We also consider integrating these variations into our methodology and find that the constraints of RVO-based [21] CBFs are consistent with those of VOCBFs. This consistency arises because velocity is treated as a state rather than a control input in the acceleration-controlled unicycle model (2). The proof is as follows. The outer normal vectors $N_{R_i,k}^{R_j}, k \in \{1, 2\}$ and the tangent directional vectors $T_{R_i,k}^{R_j}, k \in \{1, 2\}$ of RVO are identical to those of VO, with O_j replaced by R_j because RVO is utilized among robots. The primary difference between RVO and VO lies in the position of the cone apex. For $RVO_{R_i|R_j}(v_{R_i}, v_{R_j})$, the apex is at $((v_{R_i} + v_{R_j})/2)$, whereas for $VO_{R_i|R_j}(v_{R_j})$, the apex is at

v_{R_j} . The RVO-based collision avoidance constraint is similar to (20) and is expressed as follows:

$$\bigcup_{k \in \{1,2\}} (N_{R_i,k}^{R_j})^\top v_{R_i} \geq c_{R_i,k}^{R_j} \quad (30)$$

where the scalar corresponding to the k th linear constraint of $RVO_{R_i|R_j}(v_{R_i}, v_{R_j})$ is given by

$$c_{R_i,k}^{R_j} = (N_{R_i,k}^{R_j})^\top \frac{v_{R_i} + v_{R_j}}{2}, \quad k \in \{1, 2\}.$$

We can construct RVO-based CBFs as follows:

$$h_{R_i,1}^{R_j}(x) = (N_{R_i,1}^{R_j})^\top v_{R_i} - c_{R_i,1}^{R_j} = \frac{1}{2} (N_{R_i,1}^{R_j})^\top v_{R_i} \quad (31)$$

$$h_{R_i,2}^{R_j}(x) = (N_{R_i,2}^{R_j})^\top v_{R_i} - c_{R_i,2}^{R_j} = \frac{1}{2} (N_{R_i,2}^{R_j})^\top v_{R_i}. \quad (32)$$

By substituting R_j for O_j , the constraint of (31) becomes equivalent to the constraint of (22), and similarly for (32). Thus, constructing CBFs based on either VO or RVO is consistent. The primary reason is that VO-based approaches select a new velocity outside VO to achieve collision avoidance, and in the acceleration-controlled unicycle model (2), velocity is treated as a state rather than a control input. The same consistency applies to HRVO, as its cone apex is determined by a combination of VO and RVO.

In conclusion, we have demonstrated how to construct VOCBFs in the velocity space to achieve collision avoidance between robots and obstacles. Additionally, we prove that the constraints of these VOCBFs are always valid. By ensuring that at least one of the two VOCBF constraints is satisfied, safety is guaranteed. Finally, we prove that constructing CBFs based on either VO or RVO is consistent, as velocity in our robot dynamics (2) is treated as a state rather than a control input.

C. Physical Constraints on States

The states and controls of the robot are constrained within specified ranges based on the robot's physical capabilities. Constraints (10d), which address the physical limitations of control inputs, can be directly incorporated into the optimization problem (10) because control inputs are the optimization variables. However, in addition to these input constraints, the robot's velocities v and ω must also be restricted to specified ranges, i.e., $v_{\min} \leq v \leq v_{\max}$ and $\omega_{\min} \leq \omega \leq \omega_{\max}$. Since v and ω are states, rather than control inputs of (2), these state constraints cannot be directly included in the optimization problem. To address this, we propose enforcing state constraints on v and ω through CBFs. We define four CBFs to enforce the bounds as follows:

$$h_{v_{\min}}(x) = v - v_{\min}, \quad h_{\omega_{\min}}(x) = \omega - \omega_{\min} \\ h_{v_{\max}}(x) = v_{\max} - v, \quad h_{\omega_{\max}}(x) = \omega_{\max} - \omega.$$

The relative degree of these CBFs is 1. The corresponding constraints derived from these CBFs are the following:

$$L_f h_{v_{\min}}(x) + L_g h_{v_{\min}}(x)u + \mu_{v_{\min}}(h_{v_{\min}}(x)) \geq 0 \quad (33)$$

$$L_f h_{v_{\max}}(x) + L_g h_{v_{\max}}(x)u + \mu_{v_{\max}}(h_{v_{\max}}(x)) \geq 0 \quad (34)$$

$$L_f h_{\omega_{\min}}(x) + L_g h_{\omega_{\min}}(x)u + \mu_{\omega_{\min}}(h_{\omega_{\min}}(x)) \geq 0 \quad (35)$$

$$L_f h_{\omega_{\max}}(\mathbf{x}) + L_g h_{\omega_{\max}}(\mathbf{x})\mathbf{u} + \mu_{\omega_{\max}}(h_{\omega_{\max}}(\mathbf{x})) \geq 0 \quad (36)$$

where $\mu_{v_{\min}}$, $\mu_{v_{\max}}$, $\mu_{\omega_{\min}}$, and $\mu_{\omega_{\max}}$ represent different class $\mathcal{K}$ functions. By incorporating these constraints into the optimization problem, the states v and ω are effectively restricted within their specified ranges, ensuring compliance with the robot's physical limitations.

D. Controller Synthesis

In this section, we mainly demonstrate how to combine the constraints of CLFs, VOCBFs, and physical limitations on states and control inputs with the objective function to formulate an optimization problem like (10). Given that there are two VOCBF constraints associated with each obstacle, and at least one of these two constraints must be satisfied, we propose introducing integer variables to handle this requirement. Consequently, the optimization problem (10) is converted into a MIQP problem as follows:

CLF-VOCBF-MIQP:

$$\min_{(\mathbf{u}, \delta) \in \mathbb{R}^{m+4}} \frac{1}{2} \mathbf{u}^\top H \mathbf{u} + \frac{1}{2} (\mathbf{u} - \mathbf{u}_{\text{pre}})^\top R (\mathbf{u} - \mathbf{u}_{\text{pre}}) + \delta^\top P \delta \quad (37a)$$

$$\text{s.t. } L_f V_d(\mathbf{x}) + L_g V_d(\mathbf{x})\mathbf{u} + \gamma_d(V_d(\mathbf{x})) \leq \delta_d \quad (37b)$$

$$L_f V_\theta(\mathbf{x}) + L_g V_\theta(\mathbf{x})\mathbf{u} + \gamma_\theta(V_\theta(\mathbf{x})) \leq \delta_\theta \quad (37c)$$

$$L_f V_v(\mathbf{x}) + L_g V_v(\mathbf{x})\mathbf{u} + \gamma_v(V_v(\mathbf{x})) \leq \delta_v \quad (37d)$$

$$L_f V_\omega(\mathbf{x}) + L_g V_\omega(\mathbf{x})\mathbf{u} + \gamma_\omega(V_\omega(\mathbf{x})) \leq \delta_\omega \quad (37e)$$

(33), (34), (35), (36),

$$d - L_f h_{R_i,k}^{O_j}(\mathbf{x}) - L_g h_{R_i,k}^{O_j}(\mathbf{x})\mathbf{u} - \mu(h_{R_i,k}^{O_j}(\mathbf{x})) - \frac{dh_{R_i,k}^{O_j}(\mathbf{x})}{d\mathbf{x}_{O_j}} \frac{d\mathbf{x}_{O_j}}{dt} \leq G(1 - z_{j,k})$$

$$j \in \{0, \dots, M-1\}, \quad k \in \{1, 2\} \quad (37f)$$

$$\sum_{k=1}^2 z_{j,k} \geq 1, z_{j,k} \in \{0, 1\} \quad (37g)$$

$$\mathbf{u} \in \mathcal{U}, \quad \Delta \mathbf{u} \in \Delta \mathcal{U} \quad (37h)$$

$$\delta \in \mathbb{R}^4 \quad (37i)$$

where H , R , and P are predefined positive definite matrices, $\delta = [\delta_d, \delta_\theta, \delta_v, \delta_\omega]^\top$ represents the relaxation variables of CLFs, and $\mathbf{u}_{\text{pre}}$ represents the control input from the previous time step and is considered here to ensure smoother control input transitions. The integer variable $z_{j,k}$ corresponds to the k th constraint of the VOCBF for the j th obstacle, determining whether the constraint is active ($z_{j,k} = 1$) or inactive ($z_{j,k} = 0$). G is a large positive constant. The term $((dh_{R_i,k}^{O_j}(\mathbf{x}))/d\mathbf{x}_{O_j})(d\mathbf{x}_{O_j}/dt)$ accounts for the effect of dynamic obstacles and equals zero for static obstacles. Constraint (37g) ensures that at least one of the two VOCBF constraints is active for each obstacle, guaranteeing safety. If $z_{j,1} = 1$ or $z_{j,2} = 1$, then the robot navigates around the obstacle from its left or right side, respectively. If both $z_{j,1} = 1$ and $z_{j,2} = 1$, then the robot moves backward to avoid a collision. Constraints (37b)–(37e) corresponding to the CLFs

are used to guide the robot toward its destination and relaxed by relaxation variables δ to guarantee safety when they conflict with VOCBF constraints. Moreover, as VO-based methods are prone to deadlocks due to their local collision avoidance nature, combining VOCBFs with CLFs helps address this issue effectively. Weight factors in P prioritize δ_d and δ_θ over δ_v and δ_ω , as constraints (37b) and (37c) are primarily responsible for navigation. Constraints (33)–(36) restrict the robot's states v and ω to their specified ranges. Constraint (37h) ensures that control inputs $\mathbf{u}$ remain within permissible bounds and the change rate of control inputs is constrained within a specified range.

Introducing integer variables makes the optimization problem (37) relatively concise. However, solving the MIQP problem (37) can be time-consuming. To improve computational efficiency, we propose splitting the original MIQP problem into multiple suboptimization problems in the form of CLF-VOCBF-QPs. Specifically, for M obstacles, there will be 3^M suboptimization problems. Consider the case with one obstacle. Since at least one of the two VOCBF constraints must be satisfied, the original MIQP problem is split into three suboptimization problems.

- 1) Only the constraint of $h_{R_i,1}^{O_j}(\mathbf{x})$ is considered.
- 2) Only the constraint of $h_{R_i,2}^{O_j}(\mathbf{x})$ is considered.
- 3) Both constraints of $h_{R_i,1}^{O_j}(\mathbf{x})$ and $h_{R_i,2}^{O_j}(\mathbf{x})$ are considered.

All other constraints remain consistent. Since all constraints are affine in the control inputs $\mathbf{u}$, each suboptimization problem takes the form of a CLF-VOCBF-QP, which can be efficiently solved in real time. The detailed formulation for the third scenario, where both VOCBF constraints are considered, is as follows:

CLF-VOCBF-QPs:

$$\min_{(\mathbf{u}, \delta) \in \mathbb{R}^{m+4}} \frac{1}{2} \mathbf{u}^\top H \mathbf{u} + \frac{1}{2} (\mathbf{u} - \mathbf{u}_{\text{pre}})^\top R (\mathbf{u} - \mathbf{u}_{\text{pre}}) + \delta^\top P \delta \quad (38a)$$

$$\text{s.t. } (37b), (37c), (37d), (37e)$$

$$(33), (34), (35), (36), (37h), (37i)$$

$$L_f h_{R_i,k}^{O_j}(\mathbf{x}) + L_g h_{R_i,k}^{O_j}(\mathbf{x})\mathbf{u} + \frac{dh_{R_i,k}^{O_j}(\mathbf{x})}{d\mathbf{x}_{O_j}} \frac{d\mathbf{x}_{O_j}}{dt} + \mu(h_{R_i,k}^{O_j}(\mathbf{x})) \geq 0, \quad j \in \{0, \dots, M-1\}, k \in \{1, 2\} \quad (38b)$$

where constraints (38b) indicate that both constraints of $h_{R_i,1}^{O_j}(\mathbf{x})$ and $h_{R_i,2}^{O_j}(\mathbf{x})$ are considered in this suboptimization problem. The formulations for the other two scenarios, where only one of the VOCBF constraints is considered, are omitted for brevity. Since all feasible integer solutions in the original MIQP problem are covered across these suboptimization problems, the optimal solution for the original MIQP problem is obtained by solving each subproblem and selecting the one with the minimum objective function. For the case with one obstacle, after solving the three CLF-VOCBF-QPs, the

solution with the smallest objective value is chosen as the final optimal solution of the original MIQP problem (37).

When applying CLF-VOCBF-QPs to multiple obstacles, the number of suboptimization problems increases exponentially with the number of obstacles. To further improve computational efficiency, we first determine whether the feasible region of each suboptimization problem is empty or not before solving it. Specifically, we check whether

$$\mathcal{A} := \{\mathbf{u} \in \mathbb{R}^m \mid \text{s.t. (33)–(36), (37h), (38b)}\} \quad (39)$$

is empty or not. Since all constraints in $\mathcal{A}$ are affine in $\mathbf{u}$, this can be determined using linear programming (LP). Constraints of CLFs are excluded from $\mathcal{A}$ as they are relaxed through the relaxation variables and are always satisfied. Thus, only hard constraints, such as those for collision avoidance and physical limitations, are considered. If $\mathcal{A}$ is not empty, we proceed to solve the corresponding suboptimization problem (38).

As the number of suboptimization problems grows exponentially with the number of obstacles, it becomes challenging to apply the above methods in environments with a large number of obstacles. To address this, we propose reducing computational costs further using a neural network. Specifically, we design a decision network to assist the robot in determining which side of each obstacle to navigate around, and the structure of the decision network is shown in Fig. 4.

The decision network takes as input the states of M obstacles and the robot's target state, all represented in the robot's local frame. The network first performs global feature aggregation by extracting information from all obstacles using average pooling. The global obstacle information is then concatenated with the information of each individual obstacle and the robot's target state to ensure that decisions consider both local and global context. The decision network outputs the probabilities for the robot to navigate around each obstacle in three directions, i.e., $\sum p_z = 1, z \in \{0, 1, 2\}$, where p_0 represents the probability of navigating around the obstacle from the left side, p_1 is the probability of navigating around the right side, and p_2 is the probability of moving backward (rear side). For each obstacle, the robot chooses the direction corresponding to the highest probability as its decision. For example, if p_0 is the largest, then $k \in \{1\}$ in constraint (38b), meaning the robot will navigate around from the left side; if p_1 is the largest, then $k \in \{2\}$ in constraint (38b), meaning the robot will navigate around from the right side; if p_2 is the largest, then $k \in \{1, 2\}$ in constraint (38b), meaning both constraints are considered, and the robot moves backward.

With the help of the decision network, the direction for navigating around each obstacle is determined in advance. As a result, only a single suboptimization problem needs to be solved instead of 3^M suboptimization problems. This significantly reduces the computational burden, enabling the method to handle environments with a large number of obstacles efficiently.

E. Analysis of Runtime Complexity Bounds and Feasibility

In this section, we provide a detailed analysis of the runtime complexity bounds for our proposed methods and examine the feasibility of the optimization problem.

TABLE I
RUNTIME COMPLEXITY BOUNDS WITH RESPECT
TO DIFFERENT METHODS

Methods	Runtime complexity bounds ^a	
	Lower	Upper
CLF-VOCBF-MIQP	O_1	$4^M O_1$
CLF-VOCBF-QPs	$3^M O_2 + O_1$	$3^M (O_1 + O_2)$
CLF-VOCBF-QP-DecNet	O_1	O_1

^a The runtime complexity O_1 of a QP is typically $O(n^{3.5} + n^{2.5}m)$, while the runtime complexity O_2 of an LP is $O(n^2m)$, where n represents the number of optimization variables, m is the number of constraints, and M denotes the number of obstacles.

We propose three distinct methods to solve the original MIQP problem: 1) CLF-VOCBF-MIQP directly solves the MIQP problem; 2) CLF-VOCBF-QPs split the original MIQP into multiple suboptimization problems and solve each to determine the optimal solution; and 3) CLF-VOCBF-QP-DecNet utilizes a decision network to guide the process and only needs to solve one suboptimization problem. To compare the computational efficiency of these methods, we provide detailed runtime complexity bounds, as summarized in Table I, and we assume the original MIQP problem is feasible. For simplicity, we denote the runtime complexity of a QP as O_1 , and that of an LP as O_2 . For an optimization problem with n optimization variables and m constraints, O_1 is typically $O(n^{3.5} + n^{2.5}m)$, while O_2 is typically $O(n^2m)$. MIQP is generally solved using the branch-and-bound method. The lower bound of its runtime occurs when the solution of the initial relaxation problem satisfies all integer variable constraints exactly, requiring only a single QP to be solved. The upper bound, however, is determined by the total number of possible combinations of the $2M$ integer variables in (37), where each variable $z \in \{0, 1\}$. As a result, the upper bound runtime is $2^{2M} * O_1$. For CLF-VOCBF-QPs, the process involves checking feasibility as described in (39) before solving each sub-QP. The lower bound of the runtime occurs when only one suboptimization problem is feasible, and it also corresponds to the optimal solution, while the upper bound occurs when all subproblems are feasible and need to be solved. In contrast, for CLF-VOCBF-QP-DecNet, which is guided by the decision network, only one QP is solved. Thus, both the lower and upper bounds of its runtime are the same, significantly reducing computational cost.

We also provide a feasibility analysis for our approach. Since CLF constraints are relaxed by relaxation variables, the optimization problem becomes infeasible only when the collision avoidance constraints (38b) conflict with the physical limitation constraints (33), (34), (35), (36), and (37h). The feasibility of the optimization problem depends on the choice of class $\mathcal{K}$ functions and the robot's locomotion capabilities. As proposed in [4], dynamically adjusting the decay rate of CBFs can enhance feasibility, and this method can also be incorporated into our approach.

Moreover, VO-based constraints typically require $\mathbf{v}_{R_i} \notin \text{VO}_{R_i|O_j}(\mathbf{v}_{O_j})$ to ensure safety. However, this condition is often overly conservative, as safety is generally ensured based on the current distance between the robot and the obstacle, even though it may be deemed unsafe under VO-based constraints. To address this, Roncero et al. [28] suggest relaxing VO-based

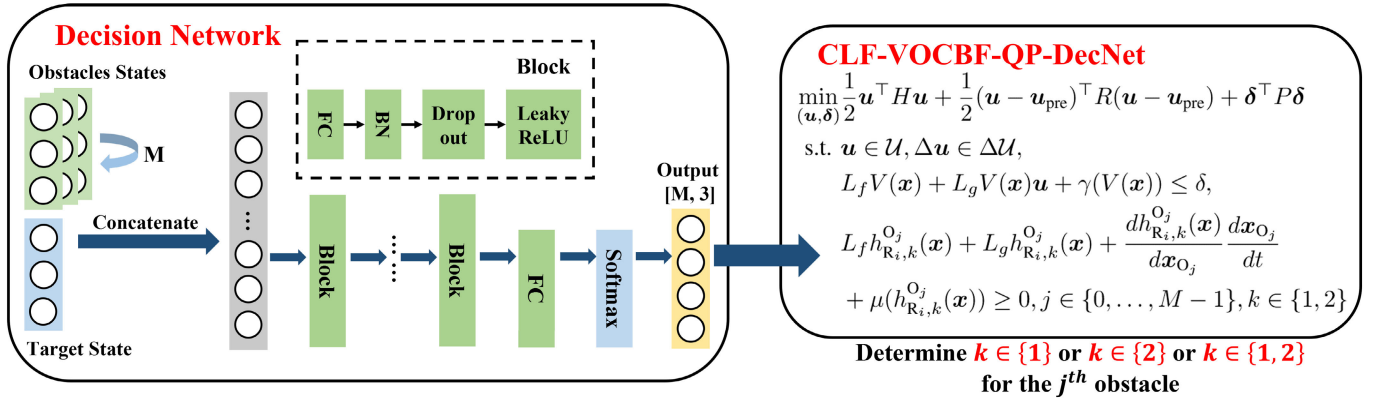


Fig. 4. Structure of the decision network. The decision network takes as input the states of M obstacles and the robot's target state, both represented in the robot's local frame. It outputs the probabilities for the robot to navigate around each obstacle in three possible directions (left, right, or backward). These probabilities are used to determine the active constraints in the formulation of the CLF-VOCBF-QP-DecNet optimization problem.

TABLE II
SETUP OF SIMULATION PARAMETERS

Notation	Meaning	Value
Δt	Time step of simulation	0.05 s
r	Radius of robot	0.3 m
l	Distance between center and rear axle axis	0.15 m
d_s	Safe margin for collision avoidance	0.15 m
$v_{\min}$	Robot's minimum linear velocity	0.0 m/s
$v_{\max}$	Robot's maximum linear velocity	4.0 m/s
$\omega_{\max}$	Robot's maximum angular velocity	0.5 rad/s
$a_{\max}$	Robot's maximum linear acceleration	1.0 m/s ²
$\alpha_{\max}$	Robot's maximum angular acceleration	0.6 rad/s ²
$\Delta a_{\max}$	Maximum change rate of linear acceleration	6.0 m/s ³
$\Delta \alpha_{\max}$	Maximum change rate of angular acceleration	3.0 rad/s ³
$\gamma(\cdot)$	Class $\mathcal{K}$ functions for all CLFs	1.0
$\mu(\cdot)$	Class $\mathcal{K}$ functions for all CBFs	1.0
$\mu_{\text{hocbf}}(\cdot)$	Class $\mathcal{K}$ functions for HOCBFs	0.75, 0.65

constraints and imposing simpler hard constraints to ensure safety, like designing the following CBF:

$$h(x) = \|p_{R_i}^{O_j}\| - d_s - \frac{v_{R_i}^{O_j}}{2a_{\max}}$$

where d_s is the safe margin. This strategy can also be applied to our approach. However, since the primary focus of this work is to introduce the construction of VOCBFs and the efficient solving of optimization problems, we refer readers to the original works for more details on these interesting methods.

In conclusion, we have demonstrated how to integrate all constraints, including CLFs, VOCBFs, and physical limitations, to formulate and efficiently solve the optimization problem. With the proposed decision network, the computational cost is significantly reduced. Furthermore, when applying our approach to distributed multirobot systems, the constraints of RVO-based CBFs are equivalent to those of VOCBFs, allowing us to incorporate the corresponding VOCBF constraints into the optimization problem.

IV. NUMERICAL SIMULATIONS

A. Implementation Details

We conducted various numerical simulations to validate the effectiveness and performance of our proposed approach. All numerical simulations are conducted using Python on an

Ubuntu Laptop with an Intel Core i9-13900HX CPU. The decision network, consisting of five blocks (as shown in Fig. 4), is trained using PyTorch on a GeForce RTX 4060 GPU. To train the decision network, we generated a dataset with 2000 random scenarios. These scenarios included randomly generated start and target positions for the robot, initial positions and velocities for two circular obstacles, and random radii for both the robot and the obstacles. Moreover, we totally propose three methods to solve the original MIQP problem: 1) CLF-VOCBF-MIQP (original form); 2) CLF-VOCBF-QPs (splitting the MIQP into multiple suboptimization problems); and 3) CLF-VOCBF-QP-DecNet (guided by the decision network, requiring only the solution of a single suboptimization problem). The parameters for the above optimization problems are identical, with class $\mathcal{K}$ functions chosen as linear scalars for simplicity. Physical constraints on the robot's kinematics are incorporated, including $v_{\min} \leq v \leq v_{\max}$, $-\omega_{\max} \leq \omega \leq \omega_{\max}$, $-a_{\max} \leq a \leq a_{\max}$, $-\alpha_{\max} \leq \alpha \leq \alpha_{\max}$, $-\Delta a_{\max} \leq \Delta a \leq \Delta a_{\max}$, and $-\Delta \alpha_{\max} \leq \Delta \alpha \leq \Delta \alpha_{\max}$. A safety margin d_s is added to the robot's radius to meet safety requirements. The simulation time step Δt is set to 0.05 s. Parameters for the robot's kinematics and the optimization controller are listed in Table II. We used BONMIN [29] to solve CLF-VOCBF-MIQP and qpOASES [30] for CLF-VOCBF-QPs and CLF-VOCBF-QP-DecNet, respectively. In Sections IV-B–IV-D, we present the simulation results and comparisons with state-of-the-art (SOTA) methods.

B. Collision Avoidance with Obstacles

In this section, we first demonstrate the effectiveness of our approach in guiding a robot to its destination while avoiding collisions with static and dynamic obstacles. We design two scenarios: one with two static obstacles and the other with two dynamic obstacles moving at constant velocities. The robot's start and target positions are the same in both scenarios, located at (0, 4 m) and (12, 10 m), respectively. The velocities of both dynamic obstacles are (−0.5, 0 m/s). Notably, the final solutions from CLF-VOCBF-MIQP and CLF-VOCBF-QPs are identical, differing only in computational efficiency. CLF-VOCBF-QP-DecNet, guided by the decision network, may occasionally yield slightly different results due to its

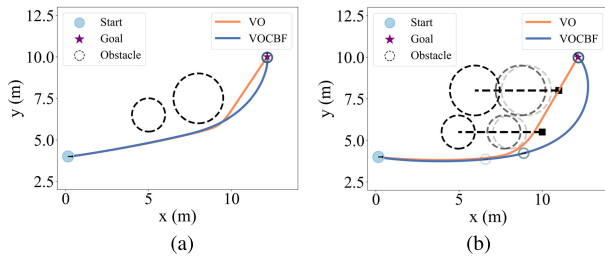


Fig. 5. Simulation results of guiding the robot to its destination while avoiding collisions with (a) static and (b) dynamic obstacles. The robot's start and target positions are represented by the light blue circle and purple star, respectively, while the black squares indicate the start positions of the dynamic obstacles. All obstacles are represented by black dashed circles. In (b), the robot's positions over time, along with those of the dynamic obstacles, are illustrated using color gradients.

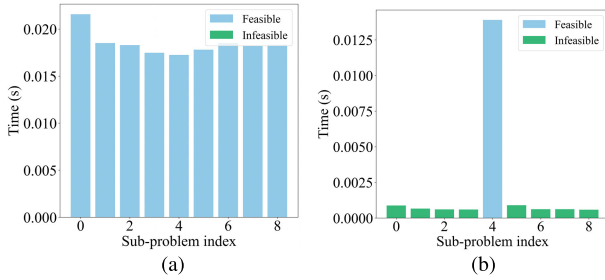


Fig. 6. Explanation of CLF-VOCBF-QPs which splits the original MIQP into multiple suboptimization problems. For two obstacles, nine subproblems are generated. To enhance efficiency, an LP is used to check if the feasible region $\mathcal{A}$ is empty before solving each problem. (a) Case 1. (b) Case 2.

TABLE III

COMPUTATION TIME OF OUR METHODS WITH RESPECT TO DIFFERENT NUMBERS OF OBSTACLES

Method	Num of obs	Time (ms)			
		Min	Max	Median	Avg
CLF-VOCBF-MIQP	1	25.3	165.9	62.9	63.1
	2	32.1	534.1	155.6	173.3
	3	36.2	2092.1	324.4	477.8
CLF-VOCBF-QPs	1	9.6	44.9	29.3	25.7
	2	13.7	178.9	46.2	72.1
	3	27.7	1223.3	193.9	309.7
CLF-VOCBF-QP-DecNet	1	8.4	12.8	9.5	9.6
	2	8.8	13.6	9.8	10.4
	3	9.2	16.8	10.6	10.8

reliance on the network. Simulation results confirm that our approach successfully navigates the robot to its destination while avoiding collisions, as shown in Fig. 5, with all three methods yielding the same results in this scenario. The robot's positions over time, along with those of the dynamic obstacles, are illustrated using color gradients.

Furthermore, to evaluate the computational efficiency of CLF-VOCBF-MIQP, CLF-VOCBF-QPs, and CLF-VOCBF-QP-DecNet, we compared their performance across varying numbers of obstacles. The runtime complexity bounds for these methods are provided in Table I, while Table III shows the computation times for each method with varying numbers of obstacles. CLF-VOCBF-QP-DecNet achieves the lowest computational cost since it is guided by the decision network and only solves a single QP problem. CLF-VOCBF-MIQP has the highest computational cost due to its reliance on solving a full MIQP problem with integer variables. CLF-VOCBF-QPs significantly reduce computational cost compared to CLF-VOCBF-MIQP by checking whether the feasible region $\mathcal{A}$

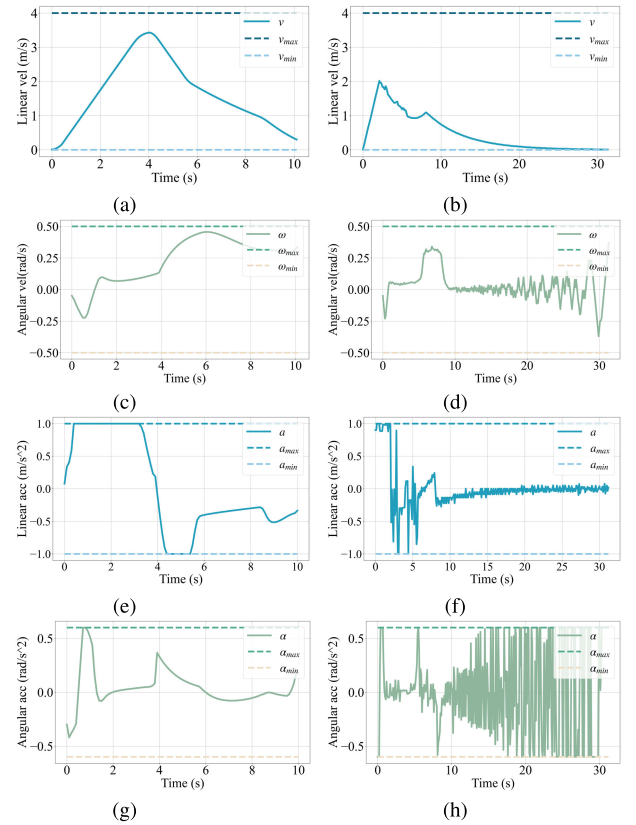


Fig. 7. Velocity and acceleration changes of VOCBF and VO. (a) VOCBF. (b) VO. (c) VOCBF. (d) VO. (e) VOCBF. (f) VO. (g) VOCBF. (h) VO.

of each suboptimization problem is empty before solving it. For example, with two obstacles, CLF-VOCBF-QPs typically solve nine suboptimization problems and select the one with the minimum objective function, as shown in Fig. 6(a). In cases like Fig. 6(b), after verifying the feasible regions, it may need to solve only one optimization problem, greatly reducing computational cost. However, CLF-VOCBF-QPs still incur higher computational cost than CLF-VOCBF-QP-DecNet due to the need for feasibility checks and solving multiple suboptimization problems. Moreover, the robot's velocity and acceleration changes in the presence of dynamic obstacles are shown in Fig. 7.

C. Compared With Benchmarks

In this section, we first compare our proposed CLF-VOCBF-QP approach with the classical VO method [12] to demonstrate that VOCBF provides stronger safety assurances. CLF-VOCBF-QP is used as it accurately represents the solution to the original MIQP problem, whereas CLF-VOCBF-QP-DecNet may occasionally deviate due to potential inaccuracies in the decision network. The test scenario involves two dynamic obstacles with constant velocities of $(-0.4, -0.15 \text{ m/s})$ and $(0.0, 0.2 \text{ m/s})$. The robot also aims to move from $(0, 4 \text{ m})$ to $(12, 10 \text{ m})$. The navigation processes for both methods are shown in Fig. 8, where the initial positions of the obstacles are the same for both methods, as indicated by the black squares. Since the robot takes different amounts of time to reach the target position in each method, the obstacles travel different distances, leading to

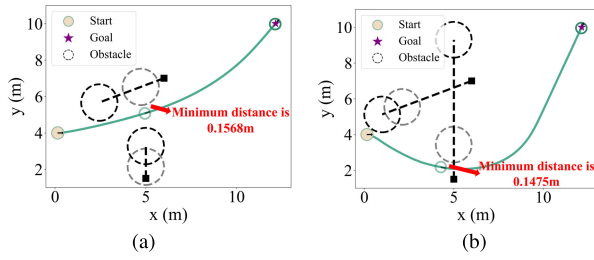


Fig. 8. Comparison of VOCBF and VO, and VOCBF provides more robust safety guarantees than VO. (a) VOCBF. (b) VO.

TABLE IV

PERFORMANCE EVALUATION WITH RESPECT TO DIFFERENT METHODS			
Methods	Evaluation Metrics (%)		
	Deadlock Rate	Completion Rate	Infeasible Rate
CLF-VOCBF-QPs	3	91	6
CLF-VOCBF-QP-DecNet	2	85	13
HOCBF	3	86	11

variations in their trajectories, as depicted in Fig. 8(a) and (b). As a result, the final positions of the obstacles are determined by the respective completion times of the robot's navigation. CLF-VOCBF-QPs, guided by CLFs, move the robot efficiently toward its target while maintaining a larger distance from obstacles than the prescribed safe margin. In contrast, VO results in a minimum robot–obstacle distance smaller than the safe margin, indicating a potential collision, as illustrated in Fig. 8(b). Additionally, VO is designed for robots controlled by horizontal and vertical velocities, which leads to oscillatory changes in states and control inputs when applied to the acceleration-controlled model (2), as shown in Fig. 7. In contrast, CLF-VOCBF-QPs ensure smoother variations in both states and control inputs, further demonstrating its effectiveness.

We also compare our proposed approach with HOCBF [17], which is constructed based on the Euclidean distance as (17), to demonstrate its better performance in dynamic obstacle avoidance. To specifically evaluate the differences in obstacle avoidance performance between VOCBF and HOCBF, all other constraints in the optimization problem remain consistent, with only the substitution of the VOCBF constraint (38b) by the corresponding HOCBF constraint. The parameters for HOCBF are chosen according to [17] and are listed in Table II.

A total of 600 random tests are conducted to evaluate the performance of CLF-VOCBF-QPs, CLF-VOCBF-QP-DecNet, and SOTA HOCBF. The evaluation is based on three key performance metrics.

- 1) *Deadlock Rate*: The percentage of cases where the robot gets stuck during navigation without collisions, indicating the method's navigation efficiency.
- 2) *Completion Rate*: The percentage of cases where the robot successfully reaches its target without collisions or deadlock, reflecting the method's overall effectiveness in collision avoidance and navigation.
- 3) *Infeasible Rate*: The percentage of cases where the optimization problem is infeasible, showing how often the method struggles with feasibility.

In the random scenario setup, the robot and two dynamic obstacles are placed in an environment bounded by (0.0, 0.0 m) and (15.0, 15.0 m). The robot's radius ranges

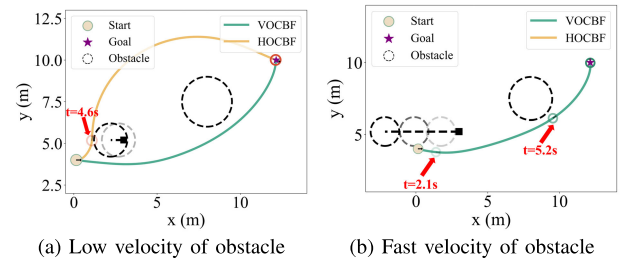


Fig. 9. Comparison between VOCBF and HOCBF. In (a), both methods can navigate the robot to its destination while avoiding collisions with obstacles. In (b), HOCBF fails to avoid collisions with the dynamic obstacle due to the optimization problem being infeasible.

TABLE V

COMPARISON BETWEEN VOCBF AND HOCBF			
Method	Scenario	Reach Time (s)	Average Solving Time (ms)
VOCBF	Slow vel.	8.1	10.5
	Fast vel.	7.9	10.6
HOCBF	Slow vel.	15.7	10.2
	Fast vel.	N/A	N/A

from 0.2 to 0.7 m, while obstacles have velocities randomly selected between (−1.0, 1.0 m/s) and radii ranging from 0.1 to 1.5 m. The performance evaluation results are presented in Table IV. CLF-VOCBF-QPs achieve the highest completion rate and the lowest infeasible rate among all methods. By splitting the MIQP problem into multiple suboptimization problems and checking feasibility before solving them, this method effectively ensures successful navigation and collision avoidance while maintaining feasibility. CLF-VOCBF-QP-DecNet has a slightly lower completion rate due to its reliance on the decision network for direction selection. Incorrect guidance from the decision network can lead to infeasibility in some scenarios, resulting in suboptimal performance. HOCBF performs worse than CLF-VOCBF-QPs, with a lower completion rate and a higher infeasible rate. This is because HOCBF, while incorporating time-varying CBFs, is fundamentally distance-based and less effective for dynamic obstacle avoidance.

Additionally, we design two scenarios to thoroughly evaluate the performance of HOCBF and VOCBF in handling dynamic obstacles. Both scenarios share the same start and target positions and include one static obstacle and one dynamic obstacle with a constant velocity. The difference lies in the velocity of the dynamic obstacle: in the first scenario, it moves slowly at (−0.05, 0.0 m/s), whereas in the second, it moves faster at (−0.6, 0.0 m/s). Both approaches employ time-varying CBFs to avoid collisions with dynamic obstacles. In this analysis, we use CLF-VOCBF-QP-DecNet, as it achieves the same results as CLF-VOCBF-QPs while requiring lower computational cost.

When the dynamic obstacle moves slowly, both methods successfully guide the robot to its destination without collisions, as shown in Fig. 9(a). Additionally, our approach demonstrates faster convergence compared to HOCBF, enabling the robot to reach its target more quickly, as demonstrated in Table V. The HOCBF-based approach requires substantial relaxation of CLFs to ensure safety over a subset of the original safe set, resulting in larger relaxation variables

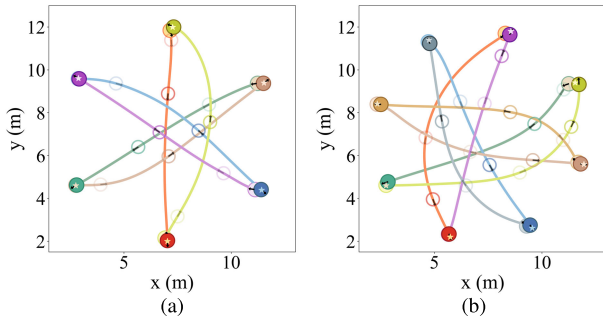


Fig. 10. Navigation of distributed multirobot systems. The positions of different robots are shown as circles with different colors. Heading angle of the robot is illustrated by an arrow within the circle. Positions over time are illustrated using color gradients. (a) Six robots. (b) Eight robots.

and a longer reach time. In contrast, CLF-VOCBF-QP-DecNet avoids excessive relaxation of CLFs, leading to a faster time. When the dynamic obstacle moves at a faster velocity, only CLF-VOCBF-QP-DecNet successfully guides the robot to its target while ensuring safety, while HOCBF fails to avoid collisions with the dynamic obstacle due to the infeasibility of the optimization problem, as shown in Fig. 9(b). This indicates that our method, which constructs VOCBF in the velocity space, is better suited for dynamic obstacle avoidance.

Remark 4: VOCBF requires the robot's velocity to lie outside VO to ensure safety. However, if the current distance between the robot and obstacle exceeds the sum of their radii while the robot's velocity falls within the VO, the situation is safe in terms of Euclidean distance but unsafe in terms of VO. This highlights the conservative nature of VO-based approaches, as VOCBF also guarantees safety over a subset of the original safe set.

D. Navigation of Distributed Multirobot Systems

In this section, we demonstrate that our proposed approach can be applied to navigation and collision avoidance in distributed multirobot systems. Each robot in the distributed multirobot systems can independently make decisions to reach its destination while avoiding collisions with others. As discussed in Section III-B, the constraints of VO-based CBFs and their variants are equivalent, enabling the application of CLF-VOCBF-QP-DecNet.

We consider the circle scenario to evaluate our method: robots are uniformly distributed on a circle of radius 5 m centered at (7, 7 m), and the initial and target positions of robots are symmetric along the center of the circle, as shown in Fig. 10(a). With CLF-VOCBF-QP-DecNet, each robot navigates independently toward its target while avoiding collisions, with robot positions at different times indicated by color gradients, as shown in Fig. 10.

In summary, while our proposed approach demonstrates promising results, challenges remain when scaling to very large multirobot systems. As the number of robots increases, the prediction accuracy of the decision network tends to decline, and the computational cost of CLF-VOCBF-QPs grows exponentially. These limitations highlight that our method requires further refinement before it can be effectively applied to large-scale robotic systems.

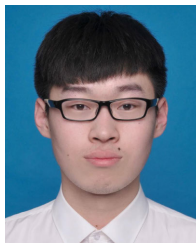
V. CONCLUSION

In this article, we propose a safety-critical controller CLF-VOCBF-QP for the acceleration-controlled unicycle model. Designing CLFs and CBFs for this model often faces the challenge that not all control inputs explicitly appear in the constraints of CLFs and CBFs. To address this issue, we propose designing state-feedback-based CLFs and constructing VOCBFs to ensure control inputs explicitly appear in the constraints. Additionally, we formulate the constraint of CLFs and VOCBFs, along with other constraints of the robot's kinematics in the form of CLF-VOCBF-MIQP. To efficiently solve the original MIQP problem, we split it into multiple suboptimization problems and employ a decision network to guide the selection and resolution of a single suboptimization problem. Numerical simulations are conducted to validate that our approach can successfully achieve navigation and collision avoidance with both static and dynamic obstacles. Furthermore, we also extend our approach to distributed multirobot systems, enabling each robot to reach its destination while avoiding collisions with other robots. Future work will focus on enhancing the accuracy of the decision network and extending our approach to large-scale multirobot systems.

REFERENCES

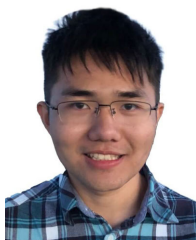
- [1] L. Ferranti, L. Lyons, R. R. Negenborn, T. Keviczky, and J. Alonso-Mora, "Distributed nonlinear trajectory optimization for multi-robot motion planning," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 2, pp. 809–824, Mar. 2023.
- [2] A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs with application to adaptive cruise control," in *Proc. 53rd IEEE Conf. Decis. Control*, Dec. 2014, pp. 6271–6278.
- [3] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.
- [4] J. Zeng, B. Zhang, Z. Li, and K. Sreenath, "Safety-critical control using optimal-decay control barrier function with guaranteed point-wise feasibility," in *Proc. Amer. Control Conf. (ACC)*, May 2021, pp. 3856–3863.
- [5] W. Xiao, C. G. Cassandras, and C. A. Belta, "Bridging the gap between optimal trajectory planning and safety-critical control with applications to autonomous vehicles," *Automatica*, vol. 129, Jul. 2021, Art. no. 109592.
- [6] A. Alan, T. G. Molnar, E. Das, A. D. Ames, and G. Orosz, "Disturbance observers for robust safety-critical control with control barrier functions," *IEEE Control Syst. Lett.*, vol. 7, pp. 1123–1128, 2023.
- [7] T. G. Molnar, A. K. Kiss, A. D. Ames, and G. Orosz, "Safety-critical control with input delay in dynamic environment," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 4, pp. 1507–1520, Jul. 2023.
- [8] J. Huang, Z. Liu, J. Zeng, X. Chi, and H. Su, "Obstacle avoidance for unicycle-modelled mobile robots with time-varying control barrier functions," in *Proc. 49th Annu. Conf. IEEE Ind. Electron. Soc.*, Jan. 2023, pp. 1–6.
- [9] J. Sun, J. Yang, and Z. Zeng, "Safety-critical control with control barrier function based on disturbance observer," *IEEE Trans. Autom. Control*, vol. 69, no. 7, pp. 4750–4756, Jul. 2024.
- [10] Y. Dong, X. Wang, and Y. Hong, "Safety critical control design for nonlinear system with tracking and safety objectives," *Automatica*, vol. 159, Jan. 2024, Art. no. 111365.
- [11] W. Xiao and C. Belta, "Control barrier functions for systems with high relative degree," in *Proc. IEEE 58th Conf. Decis. Control (CDC)*, Dec. 2019, pp. 474–479.
- [12] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *Int. J. Robot. Res.*, vol. 17, no. 7, pp. 760–772, Jul. 1998.
- [13] X. Zhang, J. Ma, Z. Cheng, M. Tomizuka, and T. H. Lee, "Velocity obstacle based risk-bounded motion planning for stochastic multi-agent systems," 2022, *arXiv:2202.09748*.

- [14] G. Wu and K. Sreenath, "Safety-critical control of a planar quadrotor," in *Proc. Amer. Control Conf. (ACC)*, 2016, pp. 2252–2258.
- [15] S. He, J. Zeng, B. Zhang, and K. Sreenath, "Rule-based safety-critical control design using control barrier functions with application to autonomous lane change," in *Proc. Amer. Control Conf. (ACC)*, 2021, pp. 178–185.
- [16] W. Xiao, C. G. Cassandras, C. A. Belta, and D. Rus, "Control barrier functions for systems with multiple control inputs," in *Proc. Amer. Control Conf. (ACC)*, Jun. 2022, pp. 2221–2226.
- [17] W. Xiao and C. Belta, "High-order control barrier functions," *IEEE Trans. Autom. Control*, vol. 67, no. 7, pp. 3655–3662, Jul. 2022.
- [18] W. Xiao, C. Belta, and C. G. Cassandras, "Adaptive control barrier functions," *IEEE Trans. Autom. Control*, vol. 67, no. 5, pp. 2267–2281, May 2022.
- [19] P. Thontepu et al., "Control barrier functions in UGVs for kinematic obstacle avoidance: A collision cone approach," 2022, *arXiv:2209.11524*.
- [20] A. D. Ames, G. Notomista, Y. Wardi, and M. Egerstedt, "Integral control barrier functions for dynamically defined control laws," *IEEE Control Syst. Lett.*, vol. 5, no. 3, pp. 887–892, Jul. 2021.
- [21] J. van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2008, pp. 1928–1935.
- [22] J. Snape, J. V. D. Berg, S. J. Guy, and D. Manocha, "The hybrid reciprocal velocity obstacle," *IEEE Trans. Robot.*, vol. 27, no. 4, pp. 696–706, Aug. 2011.
- [23] J. V. D. Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n -body collision avoidance," in *Robotics Research*. Berlin, Germany: Springer, 2011, pp. 3–19.
- [24] J. Huang, J. Zeng, X. Chi, K. Sreenath, Z. Liu, and H. Su, "Velocity obstacle for polytopic collision avoidance for distributed multi-robot systems," *IEEE Robot. Autom. Lett.*, vol. 8, no. 6, pp. 3502–3509, Jun. 2023.
- [25] H. Cheng, Q. Zhu, Z. Liu, T. Xu, and L. Lin, "Decentralized navigation of multiple agents based on ORCA and model predictive control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2017, pp. 3446–3451.
- [26] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proc. IEEE 18th Eur. Control Conf. (ECC)*, Jun. 2019, pp. 3420–3431.
- [27] T.-H. Wang, X. Wei, M. Chahine, A. Amini, R. Hasani, and D. Rus, "Learning stability attention in vision-based end-to-end driving policies," in *Proc. 5th Annu. Learn. Dyn. Control Conf.*, Jan. 2023, pp. 1099–1111.
- [28] A. Sánchez Roncero, R. I. Cabral Muchacho, and P. Ögren, "Multi-agent obstacle avoidance using velocity obstacles and control barrier functions," 2024, *arXiv:2409.10117*.
- [29] S. Sutradhar, N. B. D. Choudhury, and N. Sinha, "MINLP for hydro-thermal unit commitment problem using BONMIN solver," in *Proc. IEEE 1st Int. Conf. Power Electron., Intell. Control Energy Syst. (ICPEICES)*, Jul. 2016, pp. 1–6.
- [30] H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl, "qpOASES: A parametric active-set algorithm for quadratic programming," *Math. Program. Comput.*, vol. 6, no. 4, pp. 327–363, 2014.



Jihao Huang received the B.Eng. degree in automation from Hangzhou Dianzi University, Hangzhou, China, in 2020. He is currently pursuing the Ph.D. degree in control science and engineering with Zhejiang University, Hangzhou.

His current research interests include control theory, distributed multirobot systems, and motion planning of robots.



Jun Zeng received the B.S.E. degree from Shanghai Jiao Tong University (SJTU), Shanghai, China, in 2016, the Dipl.Ing. degree from École Polytechnique, Palaiseau, France, in 2017, and the Ph.D. degree in control and robotics from the Department of Mechanical Engineering, University of California at Berkeley, Berkeley, CA, USA, in 2022.

His research interests include the intersection of optimization, control, planning, and learning with applications on various robotics platforms.



Xuemin Chi received the B.Eng. degree in vehicle engineering from Shenyang University of Technology, Shenyang, China, in 2017, and the M.Sc. degree in vehicle engineering from Dalian University of Technology, Dalian, China, in 2019. He is currently pursuing the Ph.D. degree in control engineering with Zhejiang University, Hangzhou, China.

His research interests include motion planning and safe model predictive control algorithms.



Koushil Sreenath received the Ph.D. degree in electrical engineering and computer science from the University of Michigan, Ann Arbor, MI, USA, in 2011.

He was a Post-Doctoral Scholar with the GRASP Laboratory, University of Pennsylvania, Philadelphia, PA, USA, from 2011 to 2013, and an Assistant Professor with Carnegie Mellon University, Pittsburgh, PA, USA, from 2013 to 2017. He is currently an Associate Professor of mechanical engineering with the University of California at

Berkeley, Berkeley, CA, USA. His research interests include dynamic robotics, applied nonlinear control, and safety-critical control.

Dr. Sreenath received the NSF CAREER Award, the Hellman Fellowship Award, the Best Paper Award at the Robotics: Science and Systems (RSS), and the Google Faculty Research Award in Robotics.



Zhitao Liu (Member, IEEE) received the B.S. degree from Shandong University, Jinan, China, in 2005, and the Ph.D. degree in control science and engineering from Zhejiang University, Hangzhou, China, in 2010.

From 2011 to 2014, he was a Research Fellow with TUM CREATE, Singapore. He was an Assistant Professor from 2015 to 2016 and an Associate Professor from 2017 to 2021 with Zhejiang University, where he is currently a Professor with the Institute of Cyber-Systems and Control. He is also a Visiting

Professor at the Institute of Intelligence Science and Engineering, Shenzhen Polytechnic University. His current research interest include robust adaptive control, wireless transfer systems, and energy management systems.



Hongye Su (Senior Member, IEEE) was born in 1969. He received the B.S. degree in industrial automation from Nanjing University of Chemical Technology, Nanjing, Jiangsu, China, in 1990, and the M.S. and Ph.D. degrees in industrial automation from Zhejiang University, Hangzhou, China, in 1993 and 1995, respectively.

From 1995 to 1997, he was a Lecturer with the Department of Chemical Engineering, Zhejiang University. From 1998 to 2000, he was an Associate Professor with the Institute of Advanced Process

Control, Zhejiang University, where he is currently a Professor with the Institute of Cyber-Systems and Control. His current research interests include robust control, time-delay systems, and advanced process control theory and applications.